Instituto Tecnológico de Buenos Aires (ITBA)



25.27 - Sistemas Embebidos

Resumen 2do Parcial

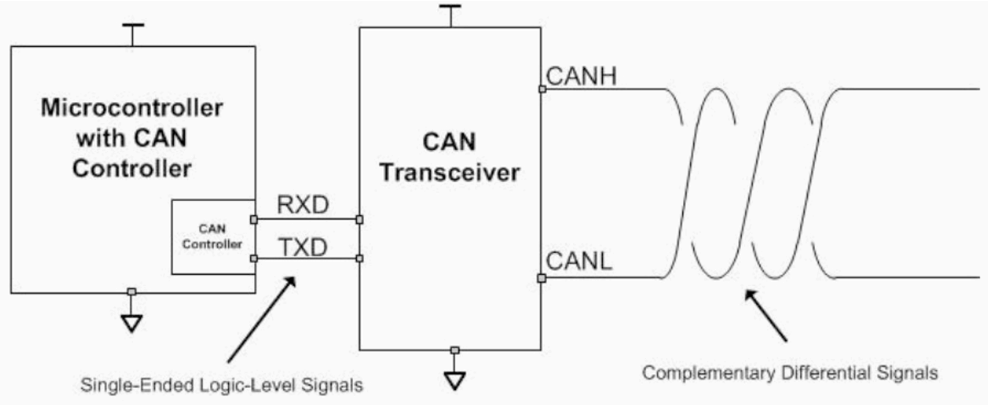
Ignacio Sammartino

Índice

Clase 7 . . . 3
Protocolo CAN (Controller Area Network) . . . 3
Arquitectura y Características Principales . . . 3
Capa Física y Señal Diferencial . . . 3
Estructura de Trama, Arbitraje y Bit Stuffing . . 4
Base de Tiempo y Sincronización (Nominal Bit Time) . . . 5
Casos de Ajuste de Flanco y Resincronización (Nominal Bit Time) . . . 5
Filtrado por Hardware y Manejo de Errores . . . 7
Clase 8 . . . 8
Periféricos de Temporización y Módulo FlexTimer (FTM) . . . 8
Fundamentos y Necesidad de Hardware Dedicado . . . 8
Arquitectura del Módulo FlexTimer (FTM) en Kinetis K64F . . . 10
Modos de Operación y Configuración por Canal . . . 13
Fuentes de Interrupción y Vectores NVIC . . . 15
Clase 9 . . . 16
ADC: Conversor Analógico–Digital . . . 16
1. Qué es y qué tiene el K64F . . . 16
2. Pasos para hacer una conversión . . . 16
3. Reloj del ADC . . . 17
4. Resolución . . . 17
5. Inicio de conversión y registro `ADCx_SC1` . . . 18
6. Lectura del resultado y flag `COCO` (EOC) . . . 18
CMP: Comparador Analógico . . . 19
Modos de operación . . . 19
PIT: Periodic Interrupt Timer . . . 21
1. Qué es y para qué sirve . . . 21
2. Clock y habilitación . . . 21
3. Registros principales . . . 21
4. Modo Chain (timers encadenados) . . . 22
5. El PIT como trigger de DMA y de ADC . . . 22
DMA: Direct Memory Access . . . 23
1. Qué es y por qué se usa . . . 23
2. Las piezas del DMA en el K64F . . . 24
3. DMAMUX: el ruteador de requests . . . 25
4. Trigger periódico: Gated vs. Always-on . . . 27
5. El eDMA y el TCD (la “orden de trabajo”) . 29
6. Generación de direcciones y tamaño de datos . . . 30
7. Minor loop y Major loop . . . 31
8. Qué pasa al terminar el major loop . . . 33
9. Habilitar el canal: `ERQ`, `SERQ` y `CERQ` . . . 33
10. Buffer circular por hardware: `SMOD` y `DMOD` . 34
11. Ejemplo completo: PWM con duty variable desde una tabla (DMA + FTM) . . . 35
12. Modos avanzados: Scatter-Gather y Channel Linking . . . 38
13. Ejemplo: buffer circular de 10 elementos (no potencia de 2) . . . 39
14. Herramienta de depuración: EmbSysRegView . . . 40
15. Resumen y checklist de configuración . . . 40
Clase 10 . . . 42
Introducción a ARM Cortex-M4: Startup y Toolchain . . . 42
CPU vs. microcontrolador . . . 42
Qué es ARM y qué define una arquitectura . . 42
Mapa de registros del Cortex-M4 . . . 44
Arquitectura del Cortex-M4 y mapa de memoria . . . 45
¿Dónde se guarda cada variable? . . . 47
Booteo del ARM Cortex-M4 . . . 49
La toolchain ARM . . . 50
Secciones y linker script . . . 52
Código de startup . . . 55
Startup real del Kinetis (`startup_mk64f12.c`) . 56
Resumen final . . . 58

Clase 7

Protocolo CAN (Controller Area Network)



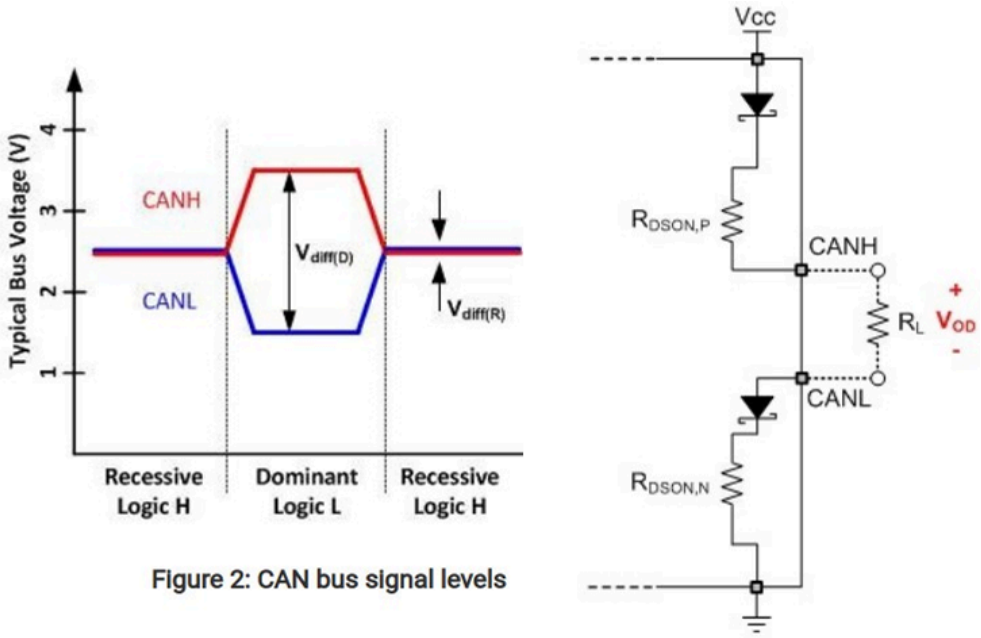


Figure 2: CAN bus signal levels

Arquitectura y Características Principales

CAN es un estándar de bus serie asíncrono, multimaestro y altamente robusto diseñado originalmente por Bosch para aplicaciones automotrices e industriales (ISO 11898). Permite que múltiples Unidades de Control Electrónico (ECUs) se comuniquen sobre un bus común sin necesidad de un nodo maestro central o controlador host.

Características fundamentales del protocolo:

- **Multimaestro / Asíncrono:** Cualquier nodo puede iniciar la transmisión cuando el bus está libre (**Idle**).
- **Orientado a Mensajes:** Los mensajes no contienen direcciones de nodo origen o destino; en su lugar, incorporan un **Identificador (ID)** que define la naturaleza del dato y su prioridad.
- **Inmunidad al Ruido:** Utiliza señalización diferencial sobre par trenzado.
- **Arbitraje No Destructivo:** Resuelve colisiones en tiempo real mediante lógica de bus dominante/recesivo (**Wired-AND**).
- **Detección y Manejo de Errores:** Lógica integrada por hardware para contención y aislamiento de fallas.

Capa Física y Señal Diferencial

A nivel eléctrico, el bus CAN se compone de un par de cables trenzados (**twisted pair**) denominados **CANH** ($\mathrm{CAN}+$ y **CANL** ($\mathrm{CAN}-$, acompañados por una referencia común de masa (GND). Para evitar reflexiones de

onda electromagnética en altas frecuencias, el bus exige **resistencias de terminación de** 120Ω instaladas en ambos extremos físicos de la línea.

El bus opera bajo dos estados eléctricos distintos que implementan la lógica **Wired-AND**:

⚙️ Estado Recesivo ('1' Lógico)

- Transceptores en alta impedancia (apagados).
- Las líneas descansan pasivamente a través de resistencias **pull-middle**.
- Voltaje: $V_{\text{CANH}} = 2.5\ \text{V}$, $V_{\text{CANL}} = 2.5\ \text{V}$.
- Tensión diferencial: $V_{\text{diff}} = V_{\text{CANH}} - V_{\text{CANL}} = 0\ \text{V}$.

⚙️ Estado Dominante ('0' Lógico)

- Transceptores activos excitando la línea.
- Se fuerzan niveles de tensión separados.
- Voltaje: $V_{\text{CANH}} = 3.5\ \text{V}$, $V_{\text{CANL}} = 1.5\ \text{V}$.
- Tensión diferencial: $V_{\text{diff}} = 2.0\ \text{V}$.

💡 Lógica Wired-AND y Colisión

Al ser un bus cableado en configuración **Wired-AND**, si dos nodos intentan transmitir simultáneamente y uno de ellos impone un estado **Dominante ('0')** mientras el otro envía un **Recesivo ('1')**, el estado dominante sobrescribe físicamente al recesivo en todo el bus. Esto permite resolver colisiones sin destruir el mensaje dominante.

Estructura de Trama, Arbitraje y Bit Stuffing

Trama de Datos Estándar (Standard Data Frame)
Un **Data Frame** Estándar (CAN 2.0A) se compone de los siguientes campos secuenciales:

Campo	Longitud	Descripción
SOF (Start of Frame)	1 bit	Bit dominante ($1 \rightarrow 0$) que marca el inicio de trama y sincroniza a todos los nodos.
Arbitration Field	12 bits	Contiene el **Identificador (ID)** de 11 bits y el bit **RTR** (0 = Data Frame, 1 = Remote Frame).
Control Field	6 bits	Incluye bits de extensión/reserva (`IDE`, `r0`) y el `DLC` (Data Length Code, 4 bits: $0..8$ bytes).
Data Field	$0..8$ bytes	Carga útil (**payload**) del mensaje ($0..64$ bits).
CRC Field	16 bits	Secuencia de verificación de redundancia cíclica de 15 bits + 1 bit `CRC Delimiter` (recesivo).
ACK Field	2 bits	`ACK Slot` (1 bit, enviado recesivo por el emisor y forzado a dominante por receptores con CRC válido) + `ACK Delimiter` (1 bit recesivo).
EOF (End of Frame)	7 bits	Fin de trama compuesto por 7 bits recesivos consecutivos.
ITM (Intermission)	3 bits	Espacio de descanso recesivo obligatorio entre tramas (**Interframe Space**).

Arbitraje No Destructivo (CSMA/CD + AMP)
Cuando el bus entra en estado pasivo (**Idle**), cualquier nodo con un mensaje pendiente puede emitir el bit `SOF`. Si dos o más nodos inician la transmisión en el mismo instante:

1. Cada nodo transmite los bits de su `ID` mientras monitorea continuamente el nivel físico del bus.
2. Si un nodo envía un bit **recesivo ('1')** en su ID pero lee un nivel **dominante ('0')** en el bus, detecta que otro mensaje en competencia tiene mayor prioridad.
3. El nodo perdedor retira su transmisor inmediatamente, pasa a modo de solo escucha (**listening only**) y aguarda a que el bus vuelva a quedar libre.

4. **Regla de prioridad:** Cuanto **menor sea el valor numérico del ID**, mayor es su prioridad en la red (el bit dominante '0' predomina en las primeras posiciones).

Regla de Bit Stuffing

Debido a que CAN es un protocolo asíncrono sin línea física de reloj, los receptores dependen de las transiciones de voltaje ($1 \rightarrow 0$) para sincronizar sus relojes internos.

> ⚠️ **Mecanismo de Bit Stuffing**
>
> Si el transmisor detecta **5 bits consecutivos de la misma polaridad** en el flujo de datos, el hardware inserta automáticamente **1 bit de relleno (Stuff Bit)** de polaridad opuesta antes de enviar el siguiente bit. Los nodos receptores remueven este bit de relleno de forma transparente antes de procesar la trama.
>
> Esta regla se aplica estrictamente desde el bit `SOF` hasta el final de la secuencia `CRC` (se excluyen el `CRC Delimiter`, el campo `ACK` y el `EOF`).

Base de Tiempo y Sincronización (Nominal Bit Time)

División del Bit en Time Quanta t_q

En redes CAN, la duración de cada bit (**Nominal Bit Time**) se subdivide en N unidades discretas de tiempo denominadas **Time Quanta** (t_q. La duración de 1 t_q se deriva del reloj interno del microcontrolador (f_{osc} mediante el divisor **Baud Rate Prescaler** (BRP.

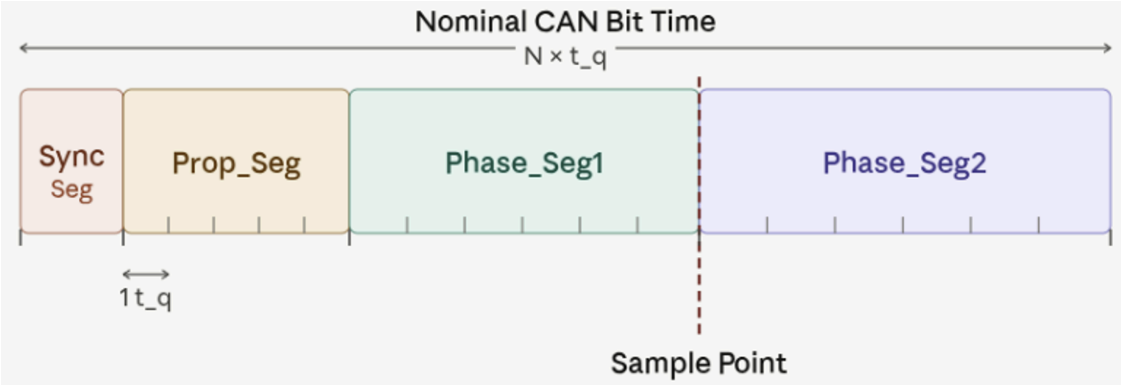


Figura 4: Estructura interna del Nominal Bit Time en sus cuatro segmentos de Time Quanta

El tiempo de bit se divide exactamente en cuatro segmentos configurables:

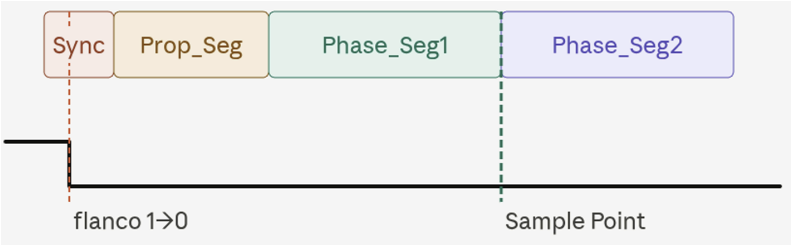
1. **Sync_Seg (Synchronization Segment):** Fijado siempre en 1 t_q. Es el intervalo donde se espera que ocurra el flanco descendente ($1 \rightarrow 0$ que marca el inicio del bit.
2. **Prop_Seg (Propagation Segment):** Configurable de 1 a 8 t_q. Compensa los retrasos físicos de propagación de las ondas electromagnéticas a lo largo del cable y los retardo internos de los transceptores.
3. **Phase_Seg1 (Phase Buffer Segment 1):** Configurable de 1 a 8 t_q. Se ubica inmediatamente antes del **Sample Point**. Se alarga en la resincronización si el flanco llega tarde.
4. **Phase_Seg2 (Phase Buffer Segment 2):** Configurable de 1 a 8 t_q. Se ubica inmediatamente después del **Sample Point**. Se acorta en la resincronización si el flanco llega temprano.

> 💡 **Sample Point (Punto de Muestreo)**
>
> El **Sample Point** es el instante exacto, situado justo al final de `Phase_Seg1`, en el que el nodo evalúa el estado lógico del bus (dominante o recesivo). Se suele configurar típicamente entre el 75% y el 87.5% de la duración total del bit.

Casos de Ajuste de Flanco y Resincronización (Nominal Bit Time)

En el protocolo CAN, la resincronización continua de los nodos receptores se realiza exclusivamente sobre los flancos descendentes ($1 \rightarrow 0$).

Caso 1: Flanco Ideal (Sin Ajuste)



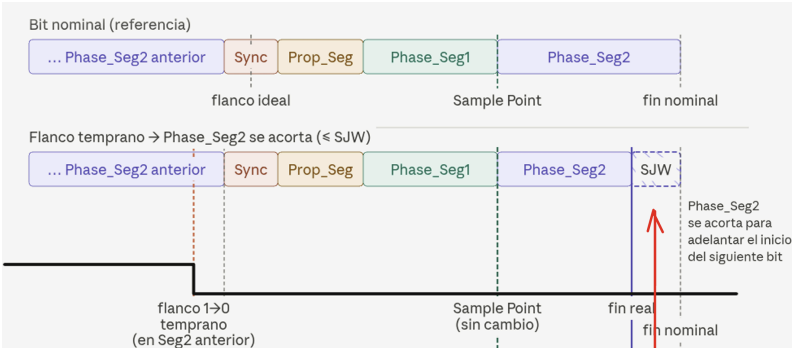
- **Ubicación del Flanco:** El flanco descendente ($1 \to 0$) llega exactamente dentro del segmento `Sync_Seg` (de duración fija $1t_q$).
- **Estado de Reloj:** El oscilador del nodo receptor está perfectamente sincronizado con el del transmisor.
- **Acción:** No se realiza ningún ajuste en la duración de los segmentos ($\Delta t_q = 0$) y el **Sample Point** permanece en su posición nominal.

Caso 2: Flanco Tardío (Reloj del Transmisor Más Lento)



- **Ubicación del Flanco:** El flanco descendente ($1 \to 0$) llega con retraso, cayendo en los segmentos `Prop_Seg` o `Phase_Seg1` en lugar de `Sync_Seg`.
- **Estado de Reloj:** El reloj del nodo transmisor es **más lento** que el del nodo receptor.
- **Mecanismo de Corrección:**
  - Se **alarga** el segmento `Phase_Seg1` agregando ciclos t_q adicionales.
  - La cantidad de ciclos agregados es igual al error de fase detectado, acotado por el valor máximo de `SJW` ($+\Delta t_q \leq \text{SJW}$).
  - **Efecto:** Se desplaza el **Sample Point** hacia la derecha (retrasando el momento de muestreo) para capturar correctamente el dato.

Caso 3: Flanco Temprano (Reloj del Transmisor Más Rápido)



- **Ubicación del Flanco:** El flanco descendente ($1 \to 0$) llega antes de lo esperado, cayendo dentro del segmento `Phase_Seg2` del bit anterior.
- **Estado de Reloj:** El reloj del nodo transmisor es **más rápido** que el del nodo receptor.
- **Mecanismo de Corrección:**
  - Se **acorta** el segmento `Phase_Seg2` substrayendo ciclos t_q de su duración.
  - La reducción está acotada por el valor máximo configurado en `SJW` ($-\Delta t_q \leq \text{SJW}$).
  - **Efecto:** Adelanta la finalización del bit actual e inicia el próximo ciclo antes de lo nominal para reengancharse con el transmisor rápido.

Caso	Ubicación del Flanco	Clock Transmisor	Ajuste de Hardware
Caso 1: Ideal	Dentro de `Sync_Seg`	Sincronizado	Sin ajuste ($\Delta t_q = 0$).
Caso 2: Tardío	En `Prop_Seg` / `Phase_Seg1`	Más lento	Se alarga `Phase_Seg1` ($+\Delta t_q \leq \text{SJW}$).
Caso 3: Temprano	En `Phase_Seg2` anterior	Más rápido	Se acorta `Phase_Seg2` ($-\Delta t_q \leq \text{SJW}$).

> ⚠️ **¡Sincronización Solo en Flancos Descendentes!**
>
> En CAN, la sincronización se realiza **exclusivamente sobre los flancos descendentes ($1 \rightarrow 0$, paso a dominante)**.
>
> Los flancos ascendentes ($0 \rightarrow 1$, paso a recesivo) **no se utilizan** debido a que la subida del bus es pasiva (depende de la resistencia **pull-middle** y capacitancias parásitas del cable), generando pendientes lentas e imprecisas.

Filtrado por Hardware y Manejo de Errores

Filtrado de Mensajes (Filter y Mask)

Para evitar sobrecargar la CPU del microcontrolador analizando mensajes no relevantes, el módulo controlador CAN (como el FlexCAN en el Kinetis K64F) filtra las tramas por hardware mediante dos registros:

- **Filter (Identificador de Filtro):** Contiene el patrón binario de ID específico que el nodo está programado para recibir.
- **Mask (Máscara de Aceptación):** Define qué bits del ID deben coincidir obligatoriamente:
  - Bit de máscara en `'1'`: **Verificación estricta** (el bit del ID entrante debe ser idéntico al bit del filtro).
  - Bit de máscara en `'0'`: **Don't Care** (se ignora la coincidencia de ese bit en particular).

Estados del Nodo y Manejo de Errores

Cada nodo CAN posee dos contadores internos de error mantenidos por hardware: `TEC` (**Transmit Error Counter**) y `REC` (**Receive Error Counter**). Según sus valores, el nodo transiciona entre tres estados de operación:

Estado del Nodo	Condición de Errores	Comportamiento en la Red
Error Active	$\text{TEC} < 128$ y $\text{REC} < 128$	Operación normal. Al detectar un error, transmite un **Active Error Flag** (6 bits dominantes), abortando la trama de toda la red.
Error Passive	$\text{TEC} \geq 128$ o $\text{REC} \geq 128$	El nodo está degradado. Transmite un **Passive Error Flag** (6 bits recesivos), evitando interrumpir la comunicación de otros nodos.
Bus Off	$\text{TEC} > 255$	El nodo se desconecta lógicamente del bus. No puede transmitir ni recibir hasta reiniciar el módulo FlexCAN.

Clase 8

Periféricos de Temporización y Módulo FlexTimer (FTM)

Fundamentos y Necesidad de Hardware Dedicado

En sistemas embebidos en tiempo real, el uso de interrupciones periódicas por software para generar o medir pulsos presenta limitaciones críticas cuando los anchos de pulso son muy angostos o cuando se requiere una precisión estricta sin sufrir la latencia de atención de la CPU.

Por esta razón, los microcontroladores incorporan bloques periféricos de **Timer por hardware**. Los dos conceptos fundamentales sobre los que se construyen estos periféricos son:

- **Output Compare (OC):** Permite programar un cambio de estado automático en un pin de salida (Toggle, Set, Clear) cuando el contador alcanza un valor predeterminado en el registro de comparación. Este evento ocurre por hardware de forma determinística, eliminando el impacto del jitter e impredecibilidad causados por la latencia de la ISR.

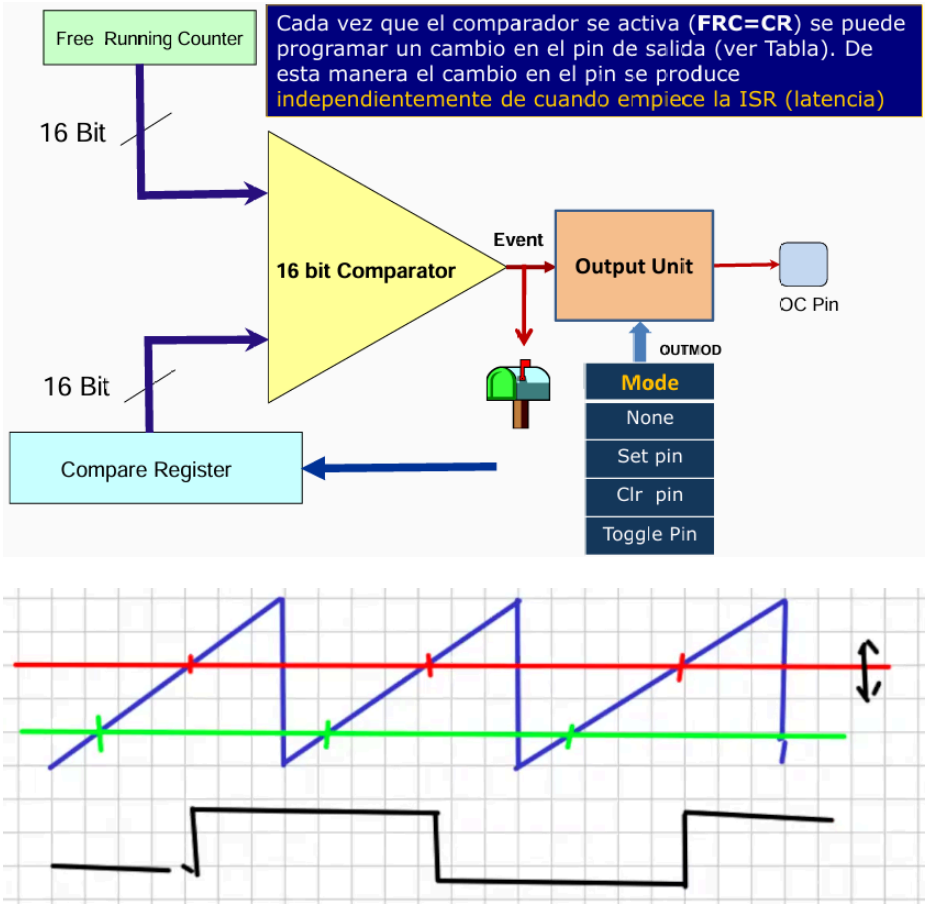


Figura 9: Ejemplo de uso con duty cycle constante

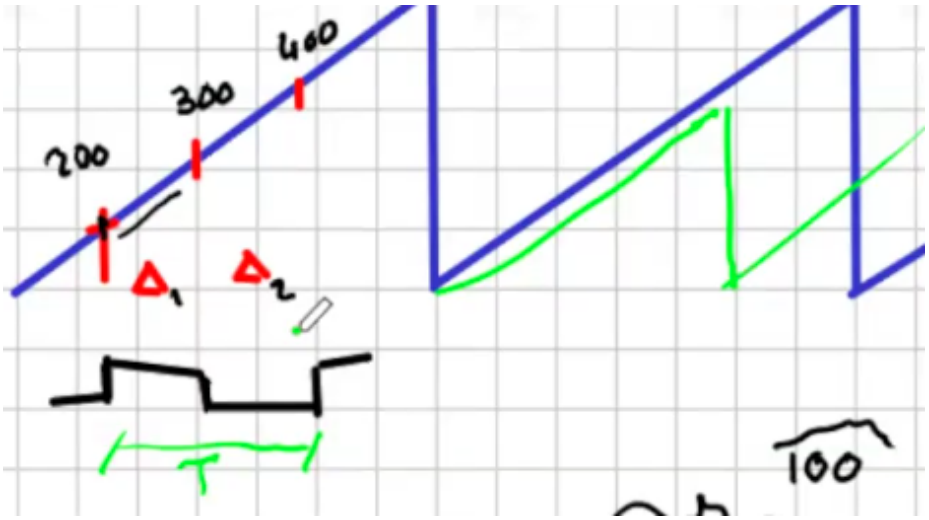


Figura 10: Ejemplo de uso con duty cycle utilizando deltas

- **Input Capture (IC):** Registra (**latch**) el valor instantáneo del contador principal ante la llegada de un flanco en un pin de entrada. Se utiliza para medir frecuencias, períodos o anchos de pulso extremadamente angostos sin sobrecargar la CPU.

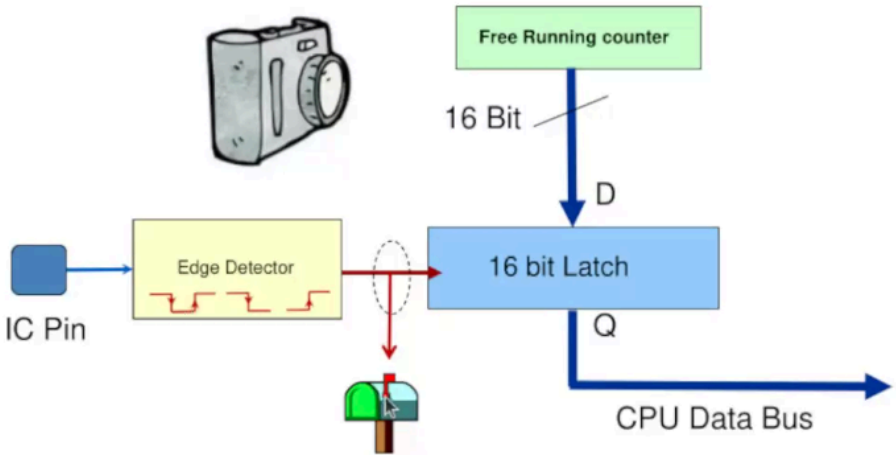


Figura 11: 'Se le saca una foto al contador en el instante del flanco de entrada'

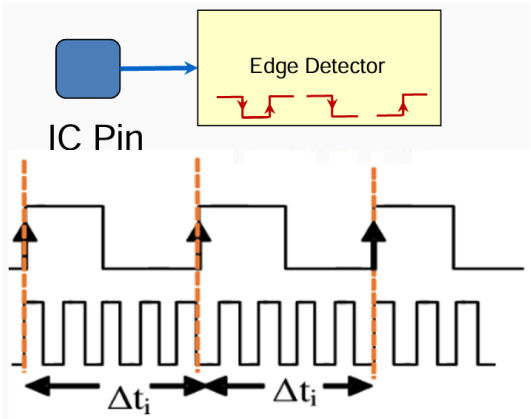


Figura 12: t_i corresponde al clock de FRC (Free Running Counter).)

> 💡 **Principio de Funcionamiento por Hardware**
>
> Ambas técnicas se basan en un contador principal de 16 bits (**Free Running Counter / FTM Counter**) accionado por un reloj del sistema [1, 11, 12]. Tanto el latch de captura como la conmutación de los pines de salida se realizan en el mismo ciclo de reloj del evento físico.

Arquitectura del Módulo FlexTimer (FTM) en Kinetis K64F

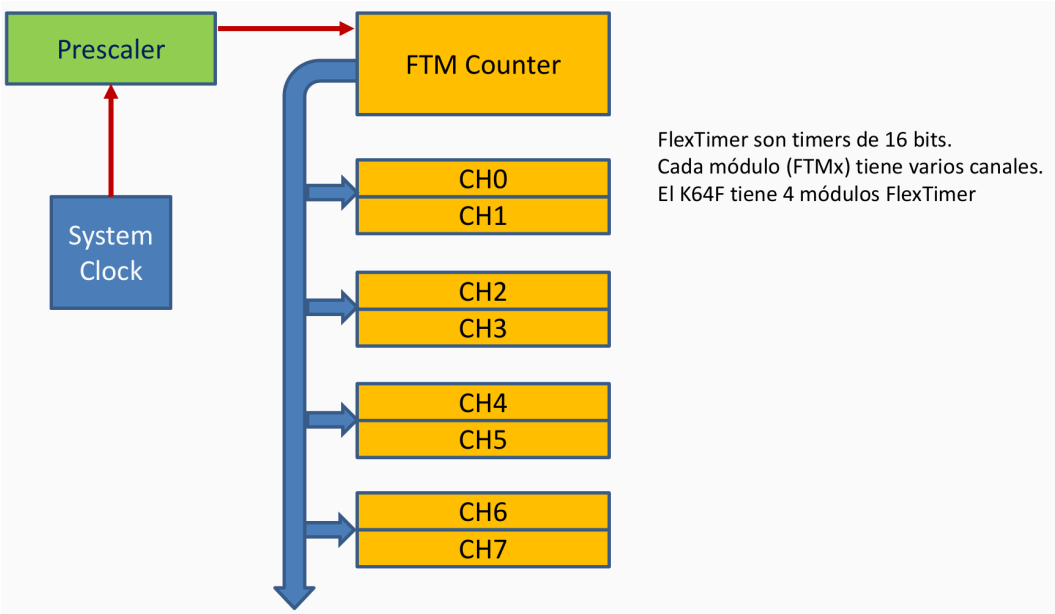


Figura 13: FTM counter y sus-submodulos.

El **FlexTimer Module (FTM)** es un temporizador multicanal de 16 bits integrado en la subfamilia Kinetis K64 (microcontrolador `MK64FN1M0VLL12` montado en la placa FRDM-K64F).

Instancias y Distribución de Canales

El microcontrolador K64F dispone de 4 instancias físicas del módulo FTM:

Instancia FTM	N° de Canales	Funciones Principales / Usos
FTM0	8 canales (CH0..CH7)	Control de motores trifásicos, PWM general, motores paso a paso.
FTM1	2 canales (CH0..CH1)	Decodificador en cuadratura (**Quadrature Decoder**) o propósito general.
FTM2	2 canales (CH0..CH1)	Decodificador en cuadratura o propósito general.
FTM3	8 canales (CH0..CH7)	Control de motores trifásicos, PWM y medición de señales.

> ⚙️ **Habilitación del Reloj (Clock Gating)**
>
> Antes de acceder a los registros de cualquier módulo FTM, es obligatorio activar la señal de reloj (**Clock Gate**) en el módulo SIM (**System Integration Module**):
> - **FTM0, FTM1, FTM2:** Se habilitan mediante los bits `SIM_SCGC6_FTM0_MASK`, `SIM_SCGC6_FTM1_MASK`, `SIM_SCGC6_FTM2_MASK` en `SIM->SCGC6`.
> - **FTM2, FTM3:** Se habilitan mediante los bits `SIM_SCGC3_FTM2_MASK` y `SIM_SCGC3_FTM3_MASK` en `SIM->SCGC3`.

Fuentes de Reloj (Clock Source) y Prescaler

El contador del FTM recibe su señal de reloj desde una llave de selección configurable en el registro de control `FTMx_SC` mediante el campo `CLKS[1:0]`:

- `00`: Sin reloj seleccionado (deshabilita el contador FTM).
- `01`: **System Clock:** Corresponde internamente al **Bus Clock** del microcontrolador ($f_{\text{bus}} = 50$ MHz en la placa FRDM-K64F a 100 MHz de Core).
- `10`: **Fixed Frequency Clock (`MCGFFCLK`):** Reloj de frecuencia fija independiente.
- `11`: **External Clock (`FTM_CLKINx`):** Reloj desde pines externos seleccionados vía `SIM_SOPT4`.

El valor del reloj seleccionado se divide posteriormente mediante un **Prescaler (PS)** de 3 bits (`PS[2:0]` en `FTMx_SC`), el cual soporta factores de división entre 1 y 128 (2^n con $n = 0..7$):

$$f_{\text{FTM_CNT}} = \frac{f_{\text{Clock_Source}}}{\text{Prescaler}}$$

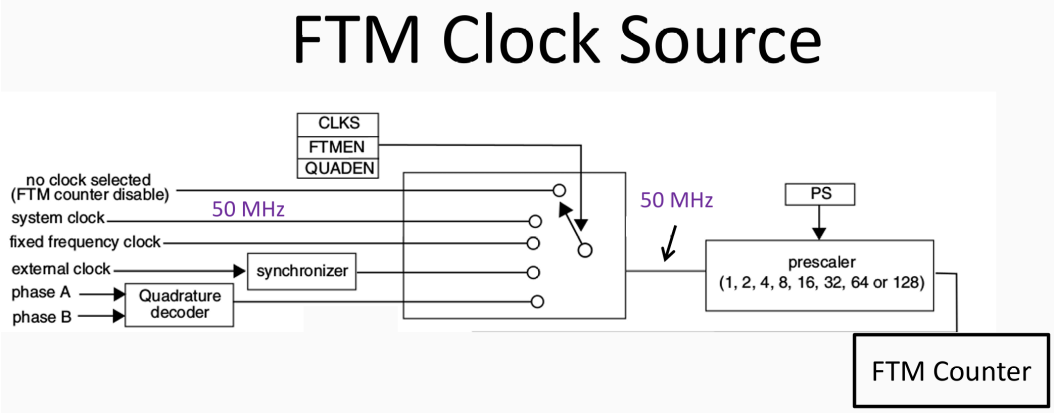


Figura 14: Clock Source

🛠️ Cálculo de Frecuencia y Resolución de Conteo

Si se selecciona el **Bus Clock** (50 MHz) y se configura un prescaler de 4 (`FTM_PSC_x4`): $f_{\text{FTM_CNT}} = \frac{50\text{ MHz}}{4} = 12.5\text{ MHz}$ Resolución de conteo (período de tick) $= \frac{1}{12.5\text{ MHz}} = 80$ ns

```
void FTM_SetPrescaler (FTM_t ftm, FTM_Prescal_t data)
{
    ftm->SC = (ftm->SC & ~FTM_SC_PS_MASK) | FTM_SC_PS(data);
}
```

En MK64F12.h

```
#define FTM_SC_PS(x)        (((uint32_t)(((uint32_t)(x))<<FTM_SC_PS_SHIFT))&FTM_SC_PS_MASK)
#define FTM_SC_PS_MASK      0x7u
#define FTM_SC_PS_SHIFT     0
```

```
typedef enum
{
FTM_PSC_x1= 0x00,
FTM_PSC_x2= 0x01,
FTM_PSC_x4= 0x02,
FTM_PSC_x8= 0x03,
FTM_PSC_x16= 0x04,
FTM_PSC_x32= 0x05,
FTM_PSC_x64= 0x06,
FTM_PSC_x128= 0x07,
} FTM_Prescal_t;
```

Ejemplo:

```
FTM_SetPrescaler(FTM0, FTM_PSC_x4); // BusClock/2=Sysclk/2= 50MHz y presccaler = divx4
```

Entonces: FTM_Coutner_Clk= 50MHz /4= 12.5MHz (Resolución 80 ns)

Modos de Conteo del Contador (`FTM Counter`)

El FTM soporta tres modos de conteo configurables mediante los registros `FTMx_SC` y `FTMx_QDCTRL`:

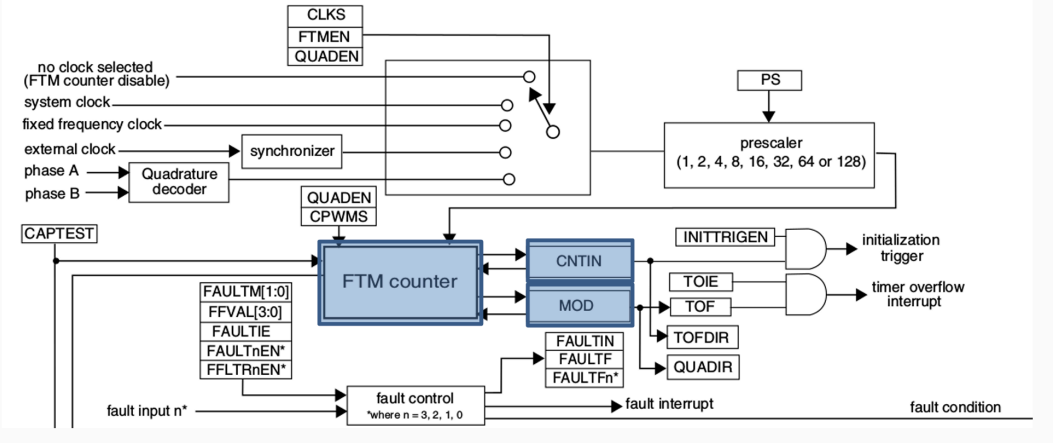


Figura 16: FTM: Up Counting

1. **Up Counting (Conteo Ascendente):**
   - **Configuración:** `QUADEN = 0`, `CPWMS = 0`.
   - **Comportamiento:** El contador se carga con `CNTIN` y se incrementa en cada tick hasta alcanzar `MOD`. Al llegar a `MOD`, se recarga automáticamente con `CNTIN` e impone `TOF = 1` (Timer Overflow Flag).
   - **Período de conteo:**

$$\text{Período} = (\text{MOD} - \text{CNTIN} + 1) \times t_{\text{FTM_CNT}}$$

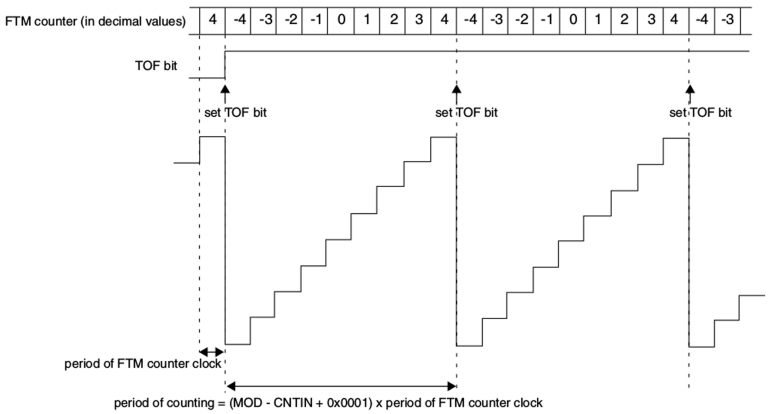


Figura 17: Up Counting, cuenta la cantidad de overflows.

2. **Up-Down Counting (Conteo Ascendente/Descendente - Center Aligned):**
   - **Configuración:** `QUADEN = 0`, `CPWMS = 1`.
   - **Comportamiento:** Cuenta desde `CNTIN` hasta `MOD`, y al llegar a `MOD` comienza a decrementar hasta regresar a `CNTIN`. El flag `TOF = 1` se activa al pasar de `MOD` a `MOD - 1`.
   - **Período de conteo:**

$$\text{Período} = 2 \times (\text{MOD} - \text{CNTIN}) \times t_{\text{FTM_CNT}}$$

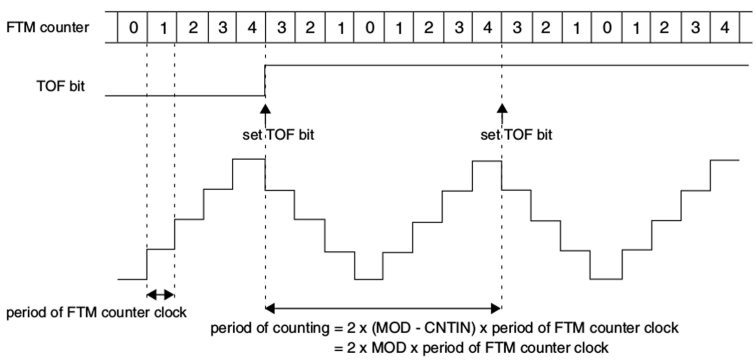


Figura 18: Up-Down Counting

3. **Free Running Counter:**

- **Comportamiento:** Se obtiene fijando `CNTIN = 0x0000` y `MOD = 0xFFFF`. El contador recorre todo el rango de 16 bits ($0..65535$) continuamente. `TOF = 1` cuando el contador pasa de 0xFFFF to 0x0000

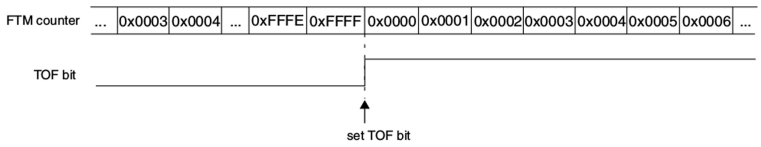


Figura 19: Free Running Counter

> ⚙️ **Registro de Filtro de Overflows (NUMTOF)**
>
> El FTM incluye el registro `NUMTOF[4:0]`, el cual especifica la cantidad de veces que el contador debe sufrir un overflow antes de que el hardware active efectivamente el bit de interrupción `TOF = 1`. Esto permite extender los intervalos de interrupción sin sobrecargar la CPU.

Modos de Operación y Configuración por Canal

Modo Compatibilidad TPM (`FTMEN = 0`) vs. Modo FTM Avanzado (`FTMEN = 1`)

El módulo FTM incluye dos modos globales de operación definidos en el registro `FTMx_MODE`:

- **FTMEN = 0 (TPM Legacy):** Modulo compatible con los timers TPM antiguos (arquitectura HCS08). Solo permite el uso del conjunto básico de registros.
- **FTMEN = 1 (FTM Advanced Functions):** Habilita todas las funciones avanzadas: Combine Mode, Deadtime Insertion, Fault Control, Quadrature Decoder, PWM Synchronization e Inversión de Salidas.

> ⚠️ **¡Protección de Escritura (WPDIS)!**
>
> Para escribir en el bit `FTMEN` o en registros avanzados, se debe desactivar primero la protección contra escritura poniendo en `'1'` el bit `WPDIS` (`Write Protection Disable`) en `FTMx_MODE`.

Modos de Trabajo Individuales y Combinados

Modo de Canal	Configuración de Bits	Descripción / Comportamiento

Input Capture (IC)	`DECAPEN=0`, `COMBINE=0`, `CPWMS=0`
	`MSnB:MSnA = 00`
	`ELSnB:ELSnA = 01` (Ascendente)
	`ELSnB:ELSnA = 10` (Descendente)
	`ELSnB:ELSnA = 11` (Ambos flancos)
		Guarda el valor instantáneo del contador `FTM_CNT` en `FTMx_CnV` al detectar el flanco configurado en el pin. Genera interrupción `CHnF = 1`.

Output Compare (OC)	`DECAPEN=0`, `COMBINE=0`, `CPWMS=0`
	`MSnB:MSnA = 01`
	`ELSnB:ELSnA = 01` (Toggle)
	`ELSnB:ELSnA = 10` (Clear)
	`ELSnB:ELSnA = 11` (Set)
		Compara `FTM_CNT` con `FTMx_CnV`. Al coincidir (`FTM_CNT == CnV`), conmuta la salida del pin automáticamente y activa `CHnF = 1`.

Edge-Aligned PWM (EPWM)	`DECAPEN=0`, `COMBINE=0`, `CPWMS=0`
	`MSnB:MSnA = 1X`
	`ELSnB:ELSnA = 10` (High-true)
	`ELSnB:ELSnA = X1` (Low-true)
		PWM alineado al borde con conteo ascendente. La salida cambia al inicio del período y al coincidir con `CnV`.
		Ancho de pulso:
		$$\mathrm{CnV} - \mathrm{CNTIN}$$
		.

Center-Aligned PWM (CPWM)	`DECAPEN=0`, `COMBINE=0`, `CPWMS=1` `ELSnB:ELSnA = 10` (High-true)	PWM centrado con conteo Up-Down. **Ancho de pulso:**

Modo de Canal	Configuración de Bits	Descripción / Comportamiento
		$2 \times (\text{CnV} - \text{CNTIN})$. **Período:** $2 \times (\text{MOD} - \text{CNTIN})$.
Combine PWM Mode	`FTMEN=1`, `COMBINE=1`, `CPWMS=0` Requiere un par de canales (n par y $n+1$).	El canal n define el borde de encendido (`CnV`) y el canal $n+1$ el borde de apagado (`C(n+1)V`). **Ancho de pulso:** $\lvert\text{C}(n+1)\text{V} - \text{CnV}\rvert$.

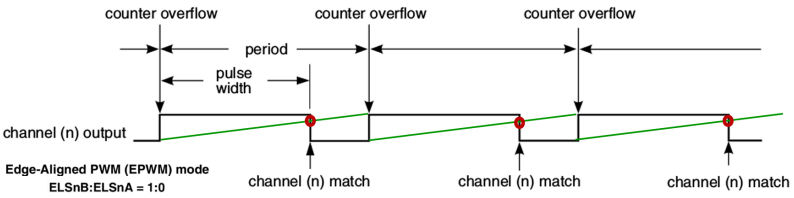


Figura 20: EPWM

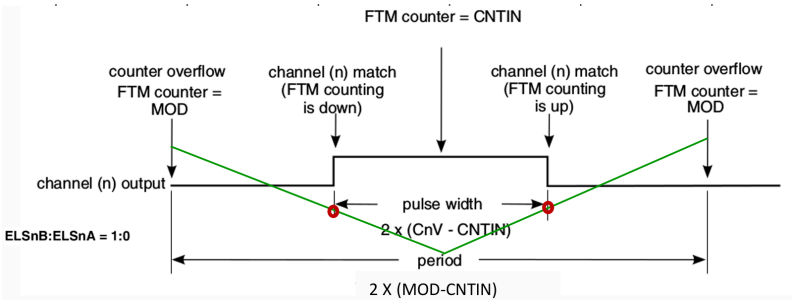


Figura 21: CPWM

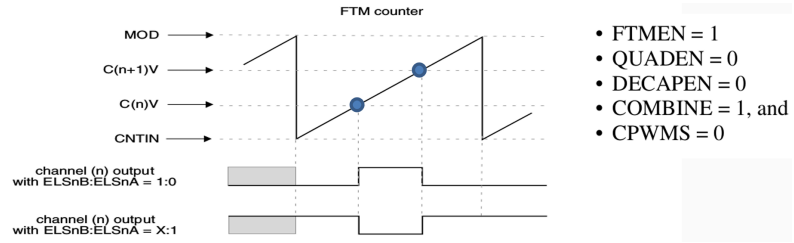


Figura 22: Combine PWM

Filtrado Digital en Captura de Entrada (Input Capture Filter)

El FTM incluye filtros digitales por hardware en los canales de entrada $0, 1, 2, 3$ para suprimir ruido o rebotes de señal.

El filtro exige que la señal permanezca estable durante una ventana temporal configurable en `FTMx_FILTER` mediante el campo `CHnFVAL[3:0]`:

$\text{Ventana de estabilidad} = 4 + 4 \times \text{CHnFVAL}[3{:}0]$ (en ciclos de clock del sistema)

Inserción de Tiempo Muerto (Deadtime Insertion)

En aplicaciones de electrónica de potencia (como el control de motores DC mediante puentes H o inversores trifásicos con MOSFETs/IGBTs), los dos conmutadores de una misma rama nunca deben conducir simultáneamente para evitar un cortocircuito directo (**shoot-through**) de la fuente de alimentación.

El FTM permite configurar salidas complementarias entre los canales n y $n+1$ (`COMP = 1` en `COMBINE`) e insertar de forma automática un tiempo muerto (**Deadtime**) a través del registro `FTMx_DEADTIME`:

⚙️ Registro FTMx_DEADTIME

- `DTPS[1:0]:` Prescaler del reloj de tiempo muerto (divisores $1, 4, 16$).
- `DTVAL[5:0]:` Número de ciclos de reloj del prescaler asignados como retardo de conmutación.

Control de Falla por Hardware (Fault Control)

El FTM monitorea hasta 4 entradas físicas de falla (`FAULTIN0..3`) para proteger las etapas de potencia. Ante una condición de sobrecorriente o sobretemperatura detectada por hardware, el módulo deshabilita inmediatamente las salidas PWM y las fuerza a un estado seguro sin esperar intervención del software.

Fuentes de Interrupción y Vectores NVIC

Cada módulo FTM consolida sus múltiples eventos de interrupción en **un único vector de interrupción** en el controlador NVIC del microcontrolador:

Fuente de Interrupción	Flags de Habilitación	Registro / Ubicación
Timer Overflow Interrupt	`TOIE = 1` y `TOF = 1`	Registro `FTMx_SC`.
Channel Interrupt (n)	`CHnIE = 1` y `CHnF = 1`	Registro de canal `FTMx_CnSC`.
Fault Interrupt	`FAULTIE = 1` y `FAULTF = 1`	Registro `FTMx_MODE`.

⚙️ Vectores IRQ en el Vector Table de K64F

- `FTM0_IRQHandler` → Vector 42 en NVIC (`FTM0_IRQn`).
- `FTM1_IRQHandler` → Vector 43 en NVIC (`FTM1_IRQn`).
- `FTM2_IRQHandler` → Vector 44 en NVIC (`FTM2_IRQn`).
- `FTM3_IRQHandler` → Vector 45 en NVIC (`FTM3_IRQn`).

Clase 9

💡 Qué vemos en esta clase

Cuatro módulos del Kinetis K64F que, combinados, permiten **adquirir y mover datos sin cargar a la CPU**:
1. **ADC**: convierte una tensión analógica en un número.
2. **CMP**: compara dos tensiones y devuelve un 0 o un 1.
3. **PIT**: timer que marca el ritmo (cada cuánto tiempo pasa algo).
4. **DMA**: mueve datos de un lugar a otro sin intervención de la CPU.

La idea final es una cadena armada **toda por hardware**: **PIT → (trigger) → ADC o DMA → memoria**.

ADC: Conversor Analógico–Digital

1. Qué es y qué tiene el K64F

Un ADC transforma una tensión (analógica) en un número entero (digital). El K64F tiene **dos módulos ADC de 16 bits** con arquitectura **SAR (Successive Approximation Register)**: el conversor va decidiendo el resultado **un bit por vez, del más significativo al menos significativo**, comparando la entrada contra una tensión generada por un DAC interno (es como una búsqueda binaria).

Cada módulo admite:

- **Hasta 24 canales Single Ended** (`AD[y]`, con `y` = número de canal): se mide la tensión de un pin respecto de masa.
- **4 canales Diferenciales** (`DAD[x][y]`, con `x` = `P` (+) o `M` (−)): se mide la diferencia entre dos pines.

🖼️ Entradas del ADC (DADP/DADM/AD), pines del K64F y selección de canal con `ADCH[4:0]` (ADCx_SC1)

💡 Notación de canales

- **Diferencial:** `DAD[x][y]`, con `x` = `M` (−) o `P` (+) e `y` = número de canal.
- **Single Ended:** `AD[y]`, equivalente a `DADP[y]` cuando `DIFF = 0`.

Cuando `DIFF = 0`, el pin `DADP[y]` pasa a funcionar como entrada Single Ended `AD[y]`.

2. Pasos para hacer una conversión

#	Qué hay que hacer	Dónde
1	Habilitar el clock del módulo ADC.	`SIM_SCGC6`
2	Elegir la fuente de reloj y el divisor.	`ADCx_CFG1` (`ADICLK`, `ADIV`)

#	Qué hay que hacer	Dónde
3	Elegir la resolución.	`ADCx_CFG1` (`MODE`)
4	Elegir el canal y modo (single/diferencial). **Escribir acá inicia la conversión.**	`ADCx_SC1[0]` (`ADCH`, `DIFF`, `AIEN`)
5	Esperar a que termine (polling del flag o interrupción).	`COCO` en `ADCx_SC1[0]`
6	Leer el resultado (esto limpia `COCO`).	`ADCx_R[0]`

3. Reloj del ADC



El ADC necesita un reloj de entrada. Se configura en `ADCx_CFG1` con dos campos:

Campo	Bits	Descripción
`ADICLK[1:0]`	`ADCx_CFG1[1:0]`	Fuente del reloj: `00` = Bus Clock; `01` = Bus Clock/2; `10` = Alternate Clock (`ALTCLK`); `11` = ADACK (reloj asincrónico del ADC).
`ADIV[1:0]`	`ADCx_CFG1[6:5]`	Divisor: `00` = ÷1; `01` = ÷2; `10` = ÷4; `11` = ÷8.

4. Resolución

La resolución se elige en `MODE[1:0]` (`ADCx_CFG1[3:2]`). En modo diferencial se agrega un bit (el signo, en complemento a 2), salvo en 16 bits.

`MODE[1:0]`	`DIFF = 0` (Single Ended)	`DIFF = 1` (Diferencial, compl. a 2)
`00`	8 bits	9 bits
`01`	12 bits	13 bits
`10`	10 bits	11 bits
`11`	16 bits	16 bits

> ⚠️ **Escribir los campos de `CFG1` juntos**
>
> Si se hacen varias asignaciones seguidas con `=` (una por campo), cada una **pisa** a la anterior. Conviene armar `CFG1` en una sola asignación (o usar `|=`):

```
SIM->SCGC6 |= SIM_SCGC6_ADC0_MASK;        // clock gate del ADC0

ADC0->CFG1 = ADC_CFG1_ADICLK(0x00)        // Bus Clock como fuente
```

```
            | ADC_CFG1_ADIV(0x00)          // sin prescaler (÷1)
            | ADC_CFG1_MODE(0x01);         // 12 bits (single ended)
```

5. Inicio de conversión y registro `ADCx_SC1`

El canal y el modo se configuran en `ADCx_SC1[0]` con el campo `ADCH[4:0]`. **Escribir en este registro inicia la conversión.**

⚙️ Registro `ADCx_SC1`: Status and Control 1

- `ADCH[4:0]`: canal de entrada. Escribirlo dispara la conversión.
- `DIFF`: `0` = Single Ended; `1` = Diferencial.
- `AIEN`: `1` = genera una interrupción al terminar la conversión (cuando `COCO = 1`).
- `COCO` (solo lectura): **Conversion Complete Flag**. Se pone en `1` al terminar. Se limpia al leer `ADCx_Rn`.

```
// Inicio de conversión por software (con polling o con interrupción)
void ADC_Start(ADC_t adc, ADCChannel_t channel, ADCMux_t mux)
{
    // MUXSEL elige cuál de los dos multiplexores de entrada (a o b) usa el canal
    adc->CFG2 = (adc->CFG2 & ~ADC_CFG2_MUXSEL_MASK) | ADC_CFG2_MUXSEL(mux);

    // AIEN: 1 = interrupción, 0 = polling. Escribir ADCH inicia la conversión.
    adc->SC1[0] = ADC_SC1_AIEN(ADC_interrupt[x]) | ADC_SC1_ADCH(channel);
}
```

6. Lectura del resultado y flag `COCO` (EOC)

Al terminar, el resultado queda en `ADCx_R[0]` y `COCO` pasa a `1`.

```
// ¿Terminó la conversión? (EOC: End Of Conversion)
bool ADC_IsReady(ADC_t adc)
{
    return adc->SC1[0] & ADC_SC1_COCO_MASK;
}

// Leer el resultado (la lectura limpia COCO)
ADCData_t ADC_getData(ADC_t adc)
{
    return adc->R[0];
}
```

⚠️ Limpieza del flag `COCO`

`COCO` se limpia solo al **leer** `ADCx_Rn`. No hay que limpiarlo a mano.

🛠️ De número a tensión

Con resolución de 12 bits (4096 valores) y una referencia de 3.3 V, la tensión de entrada es aproximadamente $V_{\text{in}} \approx \frac{\text{código}}{4096} \times 3.3\ \text{V}$. Un resultado de `2048` equivale a unos $1.65\ \text{V}$.

CMP: Comparador Analógico

El comparador **no mide** la tensión: solo dice cuál de las dos entradas es mayor. Compara la entrada positiva (`INP`) contra la negativa (`INM`) y entrega un nivel lógico (`CMPO`/`COUT`). Puede usarse en conjunto con el ADC.

Características principales:

- **DAC interno de 64 niveles**: genera la tensión de referencia contra la que se compara.
- **Histéresis configurable por hardware**: evita que la salida oscile cuando la entrada está muy cerca del umbral.
- **Varios modos de operación** (ver más abajo).



Modos de operación

Modo	Qué hace
Continuo	`COUT` refleja directamente la comparación, en modo combinacional (sin reloj).
Sampled Non-Filtered	`COUTA` se muestrea en cada flanco ascendente del reloj del bloque de filtro.
Windowed	La salida solo pasa a `COUTA` mientras la señal `WINDOW` está en alto (típicamente viene del PDB). Cuando `WINDOW = 0`, `COUTA` queda **latcheada** con el último valor.



Modo Sampled Non-Filtered

Modo Windowed: la salida solo pasa durante `WINDOW = 1` y se retiene durante `WINDOW = 0`

PIT: Periodic Interrupt Timer



1. Qué es y para qué sirve

El **PIT** es un conjunto de **4 timers de 32 bits de conteo descendente**. Sirve para:

- **Generar interrupciones periódicas** con el intervalo que se configure.
- **Proveer triggers periódicos** para el DMA (vía DMAMUX) y para el ADC (vía `SIM_SOPT7`).

Gracias a esto se puede **adquirir y transferir datos completamente por hardware**, sin que la CPU intervenga en cada muestra.

2. Clock y habilitación

El PIT funciona con el **Bus Clock** ($f_{\text{bus}} = 50\ \text{MHz}$ en la FRDM-K64F, o sea $t_{\text{bus}} = 20\ \text{ns}$):

```
SIM->SCGC6 |= SIM_SCGC6_PIT_MASK;  // Habilitar el clock gate del PIT
PIT->MCR = 0x00;                   // Encender el módulo (MDIS = 0)
```

3. Registros principales

Registro	Descripción
`PIT_MCR`	Module Control. `MDIS = 0`: módulo activo. `FRZ = 1`: los timers se frenan en modo debug.
`PIT_LDVALn`	Load Value. Define el período: el timer cuenta desde `LDVALn` hasta `0` y ahí genera la interrupción o el trigger.
`PIT_CVALn`	Current Value (solo lectura). Valor actual del contador descendente.
`PIT_TCTRLn`	Control. `TEN = 1`: habilita el timer. `TIE = 1`: habilita la interrupción. `CHN = 1`: encadena el timer n al timer $n-1$ (Chain Mode).
`PIT_TFLGn`	Flag. `TIF = 1`: el timer expiró. Se limpia **escribiendo un `1`** en ese bit.

💡 Cómo calcular `LDVAL`

El timer cuenta $\text{LDVAL} + 1$ ciclos de reloj, por eso se resta 1:

$$\text{LDVAL} = \frac{T_{\text{deseado}}}{t_{\text{bus}}} - 1$$

Ejemplo: para $T = 5.12\ \text{ms}$ con $t_{\text{bus}} = 20\ \text{ns}$:

$$\text{LDVAL} = \frac{5.12 \times 10^{-3}}{20 \times 10^{-9}} - 1 = 256000 - 1$$

4. Modo Chain (timers encadenados)

Un solo timer de 32 bits tiene un rango limitado. El **Chain Mode** lo extiende: el timer n **solo decrementa cuando el timer $n-1$ llega a cero**, así que se multiplican los períodos.

🛠️ Timer de 1 minuto con Chain Mode

Con $t_{\text{bus}} = 20\ \text{ns}$, un minuto se arma como $60\ \text{s} = 6\ \text{s} \times 10$:

- **Timer 1:** $T = 6\ \text{s} \rightarrow \text{LDVAL1} = \frac{6\ \text{s}}{20\ \text{ns}} - 1 = 3 \times 10^8 - 1$.
- **Timer 2** (con `CHN = 1`): cuenta 10 vencimientos del Timer 1 $\rightarrow \text{LDVAL2} = 10 - 1 = 9$.

```
PIT->MCR = 0x00;

// Timer 2: encadenado al Timer 1, interrupción al vencer
PIT->CHANNEL[2].LDVAL = 10 - 1;
PIT->CHANNEL[2].TCTRL = PIT_TCTRL_TIE_MASK | PIT_TCTRL_CHN_MASK | PIT_TCTRL_TEN_MASK;

// Timer 1: fuente de clock del Timer 2
PIT->CHANNEL[1].LDVAL = 300000000 - 1;
PIT->CHANNEL[1].TCTRL = PIT_TCTRL_TEN_MASK;
```

5. El PIT como trigger de DMA y de ADC

⚙️ PIT → DMA (DMAMUX)

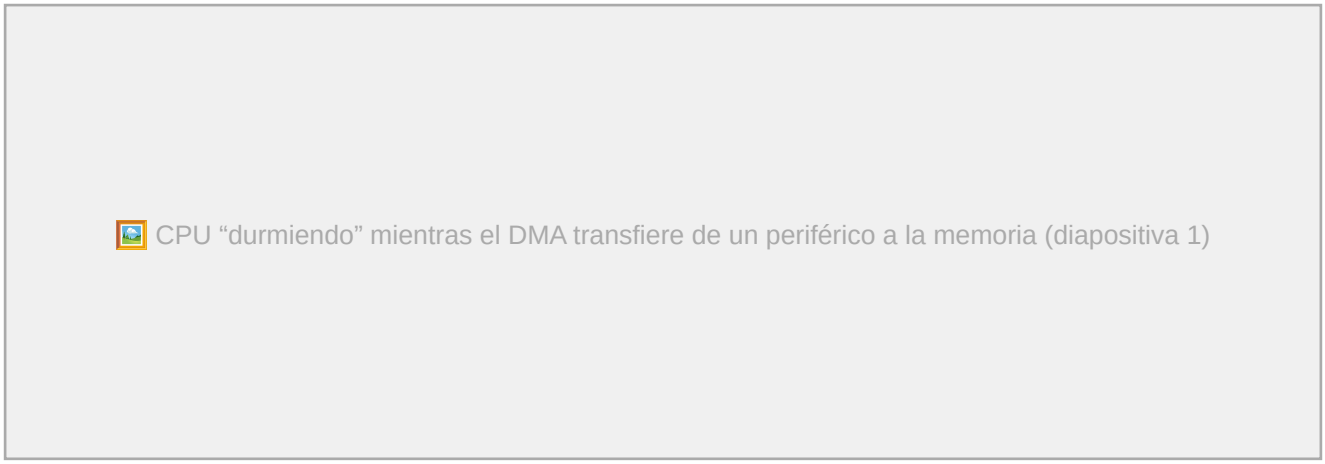
Cada canal PIT (0 a 3) puede ser el trigger periódico del canal DMA del mismo número. Se activa poniendo `TRIG = 1` en `DMAMUX_CHCFGn`.

⚙️ PIT → ADC (`SIM_SOPT7`)

El PIT puede disparar conversiones del ADC. Se configura en `SIM_SOPT7`, eligiendo el trigger y habilitando `ADCxALTTRGEN`.

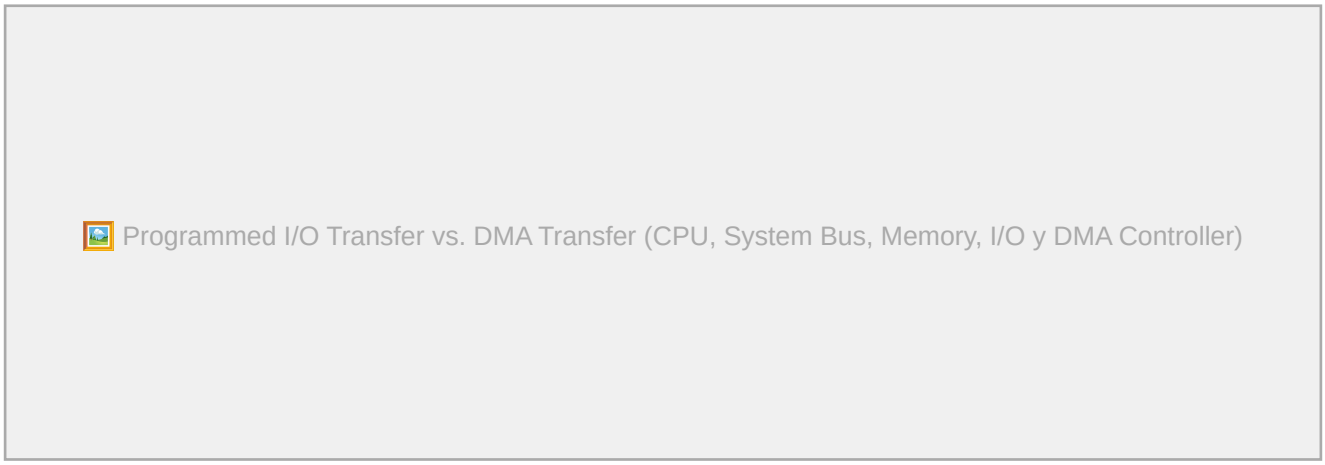
DMA: Direct Memory Access

1. Qué es y por qué se usa



El **DMA** es un controlador de hardware que **mueve datos entre memoria y periféricos** (o entre dos zonas de memoria) **sin que la CPU intervenga en cada dato**. La CPU solo configura la transferencia al principio y, si quiere, recibe un aviso al final. Mientras tanto puede hacer otras cosas.

> 💡 **Idea para quedarse**
>
> **Sin DMA** (Programmed I/O), la CPU hace **dos operaciones por cada dato**: (1) lee el periférico y (2) escribe en memoria. Si hay mil datos, son mil lecturas y mil escrituras hechas por la CPU.
>
> **Con DMA**, la CPU configura una vez “mové tantos datos de acá hasta allá” y el controlador DMA lo hace solo.



Secuencia de una transferencia DMA

#	Paso	Qué pasa
1	**Configuración**	La CPU programa los registros del controlador: dirección de origen (`From`), de destino (`To`) y cantidad de datos (`Count`).
2	**Request del periférico**	El periférico avisa que está listo para transferir (`I/O DMA Request`).
3	**Pedido del bus**	El controlador DMA le pide el bus a la CPU (`Hold Request`).
4	**La CPU acepta**	La CPU contesta (`Hold Acknowledge`) y suelta el bus (alta impedancia, `HI-Z`).

#	Paso	Qué pasa
5	**Transferencia**	El DMA lee del origen y escribe en el destino.
6	**Fin**	El contador llega a cero y el DMA avisa a la CPU con una interrupción.

> ⚠️ **Los pasos 3 y 4 son un modelo general**
>
> `Hold Request`, `Hold Acknowledge` y `HI-Z` ilustran el DMA clásico. En el K64F, el **eDMA es un maestro del bus del sistema** y su acceso se arbitra junto con el de la CPU a través del crossbar. No son pines externos del microcontrolador.

2. Las piezas del DMA en el K64F

Para entender el DMA del K64F alcanza con recordar **tres bloques** y qué hace cada uno:

Bloque	Rol
Periféricos (UART, SPI, I2C, FTM, ADC, DAC, GPIO, etc.)	**Piden** la transferencia (generan el request).
DMAMUX	**Rutea**: elige cuál de las fuentes (slots) va a cada canal del eDMA. Tiene 3 modos (disabled, normal, periodic trigger).
eDMA (controlador, 16 canales)	**Transfiere**: calcula las direcciones, lee y escribe datos. Cada canal tiene su **TCD** (la "orden de trabajo" de la transferencia).

El eDMA accede a la memoria y a los periféricos a través del **crossbar switch**, y se configura a través del **peripheral bridge**.

🖼️ eDMA Interconnections: crossbar switch, eDMA Controller, DMAMUX, peripheral bridge y periféricos (diapositiva 9)

> 🛠️ **Camino completo de un request**
>
> Un ejemplo: el **FTM0 canal 0** pide un dato → el request entra al **DMAMUX** por el slot 20 → el DMAMUX lo conecta al **canal 0 del eDMA** → el eDMA lee el **TCD del canal 0** para saber qué hacer → copia el dato desde la **memoria** hasta el **registro del FTM** a través del **crossbar**.

Clocks y habilitación (¡no olvidarse!)
Para que el DMA funcione hay que habilitar el clock de **ambos** bloques en el SIM:

Módulo	Fuente de reloj	Gating	Valor por defecto
DMA (eDMA)	System clock	`SIM_SCGC7[DMA]` (bit 1)	Habilitado (`1`)

Módulo	Fuente de reloj	Gating	Valor por defecto
DMAMUX	Bus clock	`SIM_SCGC6[DMAMUX]` (bit 1)	**Deshabilitado (`0`)**

```
SIM->SCGC6 |= SIM_SCGC6_DMAMUX_MASK;    // el DMAMUX arranca apagado: hay que prenderlo
SIM->SCGC7 |= SIM_SCGC7_DMA_MASK;       // el eDMA ya viene habilitado, pero conviene ser explícito
```


Tabla de clock source de DMA y DMAMUX, y registros `SIM_SCGC6` / `SIM_SCGC7` con los bits resaltados (diapositivas 10 y 11)

3. DMAMUX: el ruteador de requests

El **DMAMUX** conecta hasta **52 slots** (fuentes de periféricos) con los **16 canales** del eDMA. Cada canal se configura con su propio registro `DMAMUX_CHCFGn`.


Diagrama de bloques del DMAMUX: Source, Always y Trigger hacia los 16 canales del eDMA (diapositiva 12)

Tabla de slots (`SOURCE`)

El número de slot es lo que se escribe en el campo `SOURCE`. Los más usados:

Slot	Fuente	Detalle
`0`	—	Canal deshabilitado.
`2` a `11`	UART0 a UART5	`2`/`3` = UART0 RX/TX, `4`/`5` = UART1, `6`/`7` = UART2, `8`/`9` = UART3, `10` = UART4 (TX o RX), `11` = UART5 (TX o RX).
`12`, `13`	I2S0	RX / TX.
`14` a `17`	SPI0, SPI1, SPI2	`14`/`15` = SPI0 RX/TX, `16` = SPI1 (TX o RX), `17` = SPI2 (TX o RX).
`18`, `19`	I2C0, I2C1/I2C2	

Slot	Fuente	Detalle
`20` a `27`	FTM0	Canales 0 a 7.
`28`, `29`	FTM1	Canales 0 y 1.
`30`, `31`	FTM2	Canales 0 y 1.
`32` a `39`	FTM3	Canales 0 a 7.
`40`, `41`	ADC0, ADC1	
`42` a `44`	CMP0, CMP1, CMP2	
`45`, `46`	DAC0, DAC1	
`47`, `48`	CMT, PDB	
`49` a `53`	Port control (Port A a Port E)	`51` = PORTC.
`54` a `57`	IEEE 1588 Timers	Timer 0 a 3.
`58` a `63`	DMA MUX: **Always enabled**	Fuentes siempre activas.

> ⚠️ **Cantidad de slots**
>
> `SOURCE` tiene 6 bits (`SOURCE[5:0]`), por eso el número de slot puede llegar hasta 63. Las diapositivas hablan de “52 slots” de periféricos y de “10 canales always-enabled”; la tabla del Reference Manual muestra los slots `58` a `63` como always-enabled. Ante la duda, consultar la tabla **DMA request sources** del Reference Manual del K64.

Modos de operación

Modo	`ENBL` / `TRIG`	Comportamiento
Disabled	`ENBL = 0`	Canal deshabilitado. Es el estado para **configurar o reconfigurar** el canal antes de cambiar la fuente.
Normal	`ENBL = 1`, `TRIG = 0`	El slot elegido se conecta directo al canal DMA. La transferencia arranca cuando el periférico genera su request.
Periodic Trigger	`ENBL = 1`, `TRIG = 1`	El request del periférico queda **bloqueado (gated)** hasta que llega un trigger del PIT. **Solo disponible en los canales 0 a 3.**

> ⚙️ **Registro `DMAMUX_CHCFGn` (8 bits)**
>
> - `ENBL` (bit 7): `0` = canal deshabilitado (usarlo para deshabilitar o reconfigurar); `1` = canal habilitado.
> - `TRIG` (bit 6): habilita la capacidad de trigger periódico.
> - `TRIG = 0` y `ENBL = 1`: el DMAMUX solo rutea la fuente al canal (**modo Normal**).
> - `TRIG = 1` y `ENBL = 1`: modo **Periodic Trigger**.
> - `SOURCE[5:0]` (bits 5 a 0): número de slot de la fuente. Ver la tabla de slots del chip.

```
// Canal 0, fuente = SPI (slot 16), con trigger periódico del PIT
DMAMUX->CHCFG[0] = DMAMUX_CHCFG_ENBL_MASK | DMAMUX_CHCFG_SOURCE(16)
                 | DMAMUX_CHCFG_TRIG_MASK;

// Canal 0, fuente = PORTC (slot 51), modo normal (GPIO)
DMAMUX->CHCFG[0] = DMAMUX_CHCFG_ENBL_MASK | DMAMUX_CHCFG_SOURCE(51);

// Canal 0, fuente = Always-enabled (slot 63), con trigger del PIT
```

```
DMAMUX->CHCFG[0] = DMAMUX_CHCFG_ENBL_MASK | DMAMUX_CHCFG_TRIG_MASK
                 | DMAMUX_CHCFG_SOURCE(63);
```

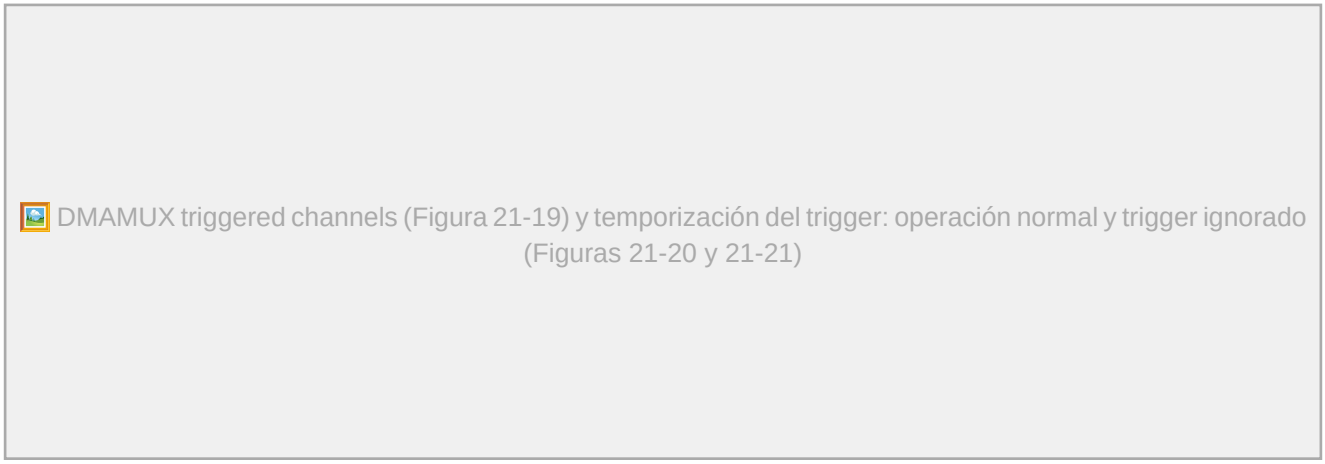
4. Trigger periódico: Gated vs. Always-on

Solo los **canales 0 a 3** pueden usar el trigger periódico, y cada uno está atado a un canal del PIT:

Canal DMA	Canal PIT
0	PIT 0
1	PIT 1
2	PIT 2
3	PIT 3

Hay dos tipos de fuente, y se comportan distinto con el trigger:

Tipo de fuente	Slots	Cómo se comporta con el PIT
Gated (periféricos)	Los slots de periféricos	**Throttled**: el request del periférico queda bloqueado hasta que llega un trigger. La transferencia se activa en el trigger **solo si el periférico está pidiendo**.
Always-enabled	Slots `58` a `63`	**Sin throttling**: el request siempre está activo, así que **cada trigger del PIT** activa una transferencia. Sirve para memoria→memoria, GPIO rápido y activación por software.



Cómo leer las figuras de temporización:

- **Operación normal** (Fig. 21-20): el request del periférico sube, pero el DMA no lo ve hasta que llega el trigger. Cuando se atiende la solicitud, el periférico baja su request.
- **Trigger ignorado** (Fig. 21-21): si llega un trigger y **el periférico no tiene request activo**, ese trigger se pierde. Cuando el periférico pida de nuevo, el DMA request queda **demorado** hasta el próximo trigger.



💡 ¿Qué modo uso?

Quiero…	Configuración
Transferir **cuando el periférico lo pida** (ej.: llegó un byte por UART).	Modo **Normal** (`TRIG = 0`) con el slot del periférico.
Transferir **a un ritmo fijo** (ej.: SPI Tx/Rx cada 5 µs).	**Periodic Trigger** (`TRIG = 1`) con el slot del periférico, canal 0 a 3 y PIT.
Generar formas de onda complejas en un GPIO desde una tabla en memoria.	Fuente **always-enabled** + **Periodic Trigger** con el PIT: una transferencia por cada tick.
Copiar memoria a memoria lo más rápido posible.	Fuente **always-enabled**, sin trigger, y arranque por software (`CSR[START]`).

Ejemplo: un pulsador que dispara el DMA

Se configura el pin `PTC6` (el pulsador SW2) para que genere un **request de DMA en flanco descendente** y se rutea ese request al canal 0:

```
/* SW2 como fuente de request DMA, en flanco descendente */
PORTC->PCR[6] |= PORT_PCR_MUX(0x01)      // GPIO
               |  PORT_PCR_IRQC(0x02)     // 0010: DMA request en flanco descendente
               |  PORT_PCR_PE_MASK        // habilita pull
               |  PORT_PCR_PS_MASK;       // pull-up

/* Canal 0 del eDMA con PORTC (slot 51) como fuente de request */
DMAMUX->CHCFG[0] |= DMAMUX_CHCFG_ENBL_MASK | DMAMUX_CHCFG_SOURCE(51);
```

El campo `IRQC` de `PORTx_PCRn` decide qué hace el pin ante un evento:

IRQC	Función
`0000`	Interrupción/DMA request deshabilitado.
`0001`	DMA request en flanco ascendente.
`0010`	DMA request en flanco descendente.
`0011`	DMA request en ambos flancos.
`1000`	Interrupción en nivel lógico 0.
`1001`	Interrupción en flanco ascendente.
`1010`	Interrupción en flanco descendente.

IRQS	Función
1011	Interrupción en ambos flancos.
1100	Interrupción en nivel lógico 1.

5. El eDMA y el TCD (la “orden de trabajo”)

El eDMA tiene dos partes:

- **eDMA Engine**: hace el trabajo (calcula las direcciones de origen y destino y transfiere los datos).
- **TCD memory** (Transfer Control Descriptor): memoria local que se ve como un **arreglo de estructuras TCD**. **Cada elemento del arreglo corresponde a un canal**. Cada TCD ocupa **32 bytes** (8 palabras de 32 bits) y describe completamente la transferencia de ese canal.



El TCD del canal i está en `0x4000_8000 + 0x1000 + 32 * i` (con i de 0 a 15).

Mapa de memoria y estructura del TCD

Offset	Campo	Qué es
0x00	SADDR (32 bits)	Source Address: dirección de origen.
0x04	SOFF (16 bits)	Source Offset: cuánto se suma a SADDR después de cada transferencia.
0x06	ATTR (16 bits)	Transfer Attributes: tamaño de dato (SSIZE/DSIZE) y módulo (SMOD/DMOD).
0x08	NBYTES (32 bits)	Minor Byte Count: bytes por cada request.
0x0C	SLAST (32 bits)	Last Source Address Adjustment: ajuste de SADDR al terminar el major loop.
0x10	DADDR (32 bits)	Destination Address: dirección de destino.
0x14	DOFF (16 bits)	Destination Offset: cuánto se suma a DADDR después de cada transferencia.
0x16	CITER (16 bits)	Current Major Iteration Count (y bits de linking).
0x18	DLASTSGA (32 bits)	Last Destination Address Adjustment / Scatter-Gather Address.
0x1C	CSR (16 bits)	Control and Status.
0x1E	BITER (16 bits)	Beginning Major Iteration Count (y bits de linking).

> ⚠️ **Ojo con el orden de los campos**

La tabla de la diapositiva 26 muestra algunos pares de campos de 16 bits intercambiados (`ATTR`/`SOFF`, `DOFF`/`CITER` y `BITER`/`CSR`). El orden correcto es el del diagrama de bits de la diapositiva 27, el de las direcciones de registro (por ejemplo `SOFF` en `+1004h` y `ATTR` en `+1006h`) y el de la estructura en C, que es el que se muestra acá.

```c
/* Estructura con los campos del TCD (32 bytes) */
typedef struct {
    uint32_t SADDR;          // Source Address (dirección de origen)
    uint16_t SOFF;           // Source Offset (offset por transferencia)
    uint16_t ATTR;           // Transfer Attributes (tamaño, SMOD, DMOD)
    union {
        uint32_t NBYTES_MLNO;       // Minor loop mapping deshabilitado
        uint32_t NBYTES_MLOFFNO;    // Mapping habilitado, sin offset de minor loop
        uint32_t NBYTES_MLOFFYES;   // Mapping habilitado, con offset de minor loop
    };
    uint32_t SLAST;          // Ajuste de SADDR al finalizar el major loop
    uint32_t DADDR;          // Destination Address (dirección de destino)
    uint16_t DOFF;           // Destination Offset (offset por transferencia)
    union {
        uint16_t CITER_ELINKNO;     // Contador actual del major loop (sin linking)
        uint16_t CITER_ELINKYES;    // Idem, con linking
    };
    uint32_t DLASTSGA;       // Ajuste de DADDR / puntero al siguiente TCD (S&G)
    uint16_t CSR;            // Control and Status (INTMAJOR, ESG, DREQ, etc.)
    union {
        uint16_t BITER_ELINKNO;     // Contador inicial del major loop (sin linking)
        uint16_t BITER_ELINKYES;    // Idem, con linking
    };
} TCD_t;
```

💡 Los campos del TCD, como preguntas

Un TCD responde a estas preguntas:

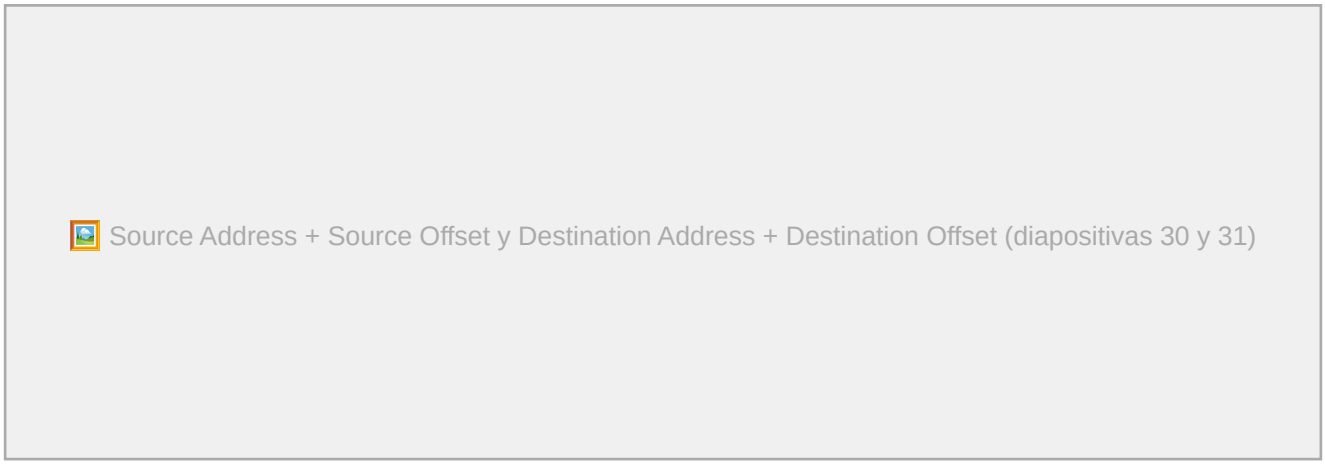
- **¿De dónde saco los datos?** `SADDR`.
- **¿A dónde los llevo?** `DADDR`.
- **¿Cómo avanzo en cada lado después de cada dato?** `SOFF` (origen) y `DOFF` (destino).
- **¿De qué tamaño es cada dato?** `ATTR` (`SSIZE`, `DSIZE`).
- **¿Cuántos bytes muevo por cada request?** `NBYTES`.
- **¿Cuántos requests atiendo en total?** `CITER` y `BITER`.
- **¿Qué hago al terminar todo?** `SLAST`, `DLASTSGA` y `CSR`.

6. Generación de direcciones y tamaño de datos

Después de cada dato transferido, el eDMA actualiza cada dirección sumándole su offset:

$$\text{SADDR}_{\text{siguiente}} = \text{SADDR} + \text{SOFF} \qquad \text{DADDR}_{\text{siguiente}} = \text{DADDR} + \text{DOFF}$$

- **Offset = 0**: la dirección **no cambia**. Sirve para leer siempre el mismo registro de un periférico, o para escribir siempre en el mismo registro.
- **Offset = tamaño del dato**: se recorre una tabla en memoria, elemento por elemento.



El tamaño de cada dato se elige por separado para origen (`SSIZE`) y destino (`DSIZE`) en `ATTR`:

`SSIZE` / `DSIZE`	Tamaño
`000`	8 bits
`001`	16 bits
`010`	32 bits
`100`	16 bytes
`101`	32 bytes
`011`, `110`, `111`	Reservados

🛠️ Recorrer un arreglo de palabras de 32 bits hacia un registro fijo

- `SSIZE = DSIZE = 2` (32 bits).
- `SOFF = 4`: avanza al siguiente elemento del arreglo.
- `DOFF = 0`: el destino (el registro) no cambia.
- `NBYTES = 4`: una palabra por cada request.

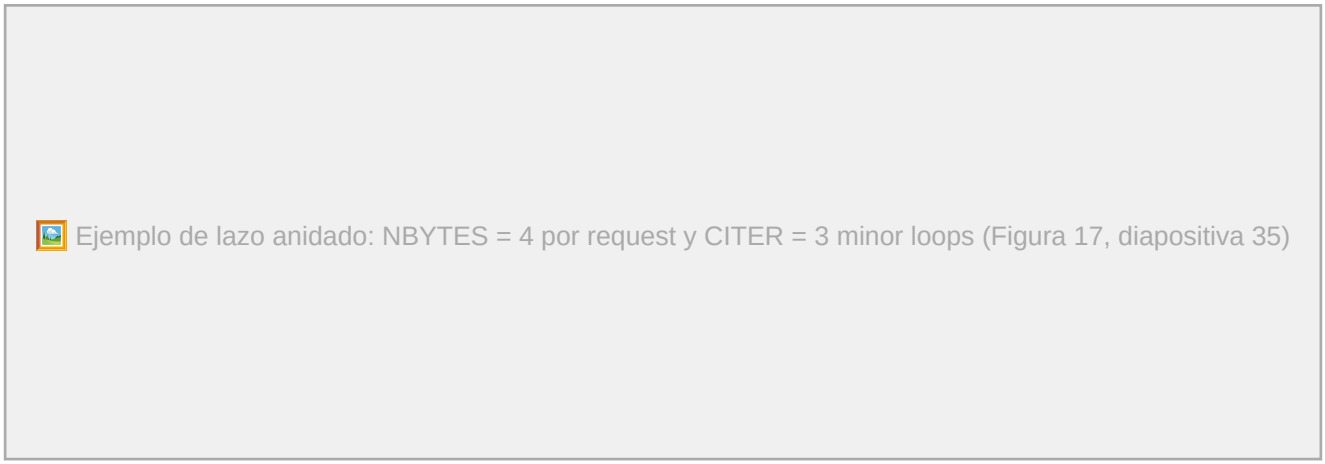
7. Minor loop y Major loop

Son las dos “vueltas” del DMA, y confunden bastante. Pensalo así:

💡 Minor loop vs. Major loop

- **Minor loop** = **lo que se mueve en un solo request** (`NBYTES` bytes). Es **indivisible**: una vez que arranca no se puede interrumpir.
- **Major loop** = **todos los requests de la transferencia** (`CITER` minor loops). Cuando se completa, la transferencia terminó.

Ejemplo: `NBYTES = 4` y `CITER = 3` significa **3 requests de 4 bytes cada uno = 12 bytes en total**.



Qué pasa en cada request (minor loop)
El eDMA recorre los `NBYTES` bytes del request (internamente `NBYTES` va decrementando). Cuando llega a `0`, actualiza el TCD: avanza `SADDR` y `DADDR` y decrementa el contador del major loop (`CITER--`).

> 🛠️ **`NBYTES` es bytes, no cantidad de elementos**
>
> `NBYTES = K × tamaño del dato`. Con datos de 32 bits (`SSIZE = DSIZE = 2`):
> - `NBYTES = 64` → 16 elementos de 32 bits por request.
> - `NBYTES = 16` → 4 elementos de 32 bits por request.

> ⚠️ **Valor de `NBYTES`**
>
> `NBYTES = 0` **no** significa “no transferir nada”: el eDMA lo interpreta como una transferencia de **4 GB** (2^{32} bytes). Programar siempre un valor válido.

Contadores `CITER` y `BITER`
- `CITER` (**Current** Major Iteration Count): cuántos minor loops **faltan**. Se decrementa al completar cada request.
- `BITER` (**Beginning** Major Iteration Count): el valor **inicial**. Cuando `CITER` llega a `0`, se recarga `CITER` desde `BITER`.

> ⚠️ **`CITER` y `BITER` deben ser iguales al configurar**
>
> Al cargar el TCD por primera vez, `CITER` y `BITER` **tienen que tener el mismo valor**. Si el canal va a hacer un único request, ambos valen `1`.

`ELINK`	Ancho del contador
`ELINK = 0` (sin channel linking)	`CITER`/`BITER` usan hasta **15 bits** (bits 14 a 0). El bit 15 es `ELINK`.
`ELINK = 1` (con channel linking en el minor loop)	Parte del campo se usa como `LINKCH` (canal enlazado) y quedan **9 bits** para el conteo.

`NBYTES` y Minor Loop Mapping
La forma del campo `NBYTES` depende del bit `CR[EMLM]` (Enable Minor Loop Mapping) del eDMA:

Variante	Condición	Qué contiene
`NBYTES_MLNO`	`EMLM = 0`	Los 32 bits son `NBYTES`.
`NBYTES_MLOFFNO`	`EMLM = 1`, `SMLOE = 0`, `DMLOE = 0`	Bits 31 y 30 = `SMLOE`/`DMLOE`; el resto es `NBYTES`. **No se aplica offset** de minor loop.

Variante	Condición	Qué contiene
`NBYTES_MLOFFYES`	`EMLM = 1` y (`SMLOE` o `DMLOE` = 1)	Además de `NBYTES` (10 bits) aparece `MLOFF` (offset de minor loop, con signo). **Sí se aplica el offset**.

`MLOFF` se suma a la dirección de origen (si `SMLOE = 1`) o a la de destino (si `DMLOE = 1`) **cada vez que termina un minor loop**. Es independiente de `SOFF` y `DOFF`, que se aplican **después de cada elemento**.

8. Qué pasa al terminar el major loop

Cuando `CITER` llega a `0`, el eDMA hace los ajustes finales:

- Suma `SLAST` a `SADDR`. Es un valor **con signo**: con un valor negativo se vuelve al principio de la tabla (por ejemplo `SLAST = -sizeof(buffer)`). También sirve para pasar a la próxima estructura de datos.
- Si `ESG = 0`, suma `DLASTSGA` a `DADDR` (igual que `SLAST` pero para el destino).
- Si `ESG = 1`, `DLASTSGA` es la **dirección del próximo TCD** (modo Scatter-Gather, más adelante).
- Recarga `CITER` desde `BITER`.
- Marca `DONE`, y puede generar una interrupción y/o activar otro canal.

> 🛠️ **Canal que repite indefinidamente una tabla**
>
> Si `SLAST = -sizeof(tabla)` y `DREQ = 0`, al terminar el major loop `SADDR` vuelve al inicio de la tabla y el canal sigue atendiendo requests: la tabla se repite una y otra vez.

Registro `CSR`: Control and Status

Campo	Función
`BWC`	Band Width Control: puede frenar al eDMA (ninguno, 4 u 8 ciclos de CPU) para repartir el ancho de banda del bus.
`MAJORLINKCH`	Número del canal a activar **al terminar el major loop** (channel linking).
`MAJORELINK`	Habilita el channel linking al terminar el major loop.
`DONE`	Flag: el eDMA completó el major loop (`CITER` llegó a `0`).
`ACTIVE`	Flag: el canal está ejecutando una transferencia (se limpia cuando termina el minor loop).
`ESG`	Habilita el modo Scatter-Gather.
`DREQ`	Disable Request: deshabilita el request del canal al terminar el major loop.
`INTHALF`	Interrupción cuando `CITER` llega a la mitad del conteo.
`INTMAJOR`	Interrupción cuando termina el major loop (`CITER` llega a `0`).
`START`	Arranca el canal explícitamente por **software** (sin esperar un request de periférico).

`DONE` y `ACTIVE` ayudan a verificar el avance del canal durante la depuración.

9. Habilitar el canal: `ERQ`, `SERQ` y `CERQ`

Hay **dos pasos distintos**, y es común olvidarse del segundo:

1. Elegir la fuente en `DMAMUX_CHCFGn` (qué request entra al canal).
2. **Habilitar las solicitudes del canal en el eDMA** (bit del canal en `DMA_ERQ`).

`DMA_ERQ` tiene un bit por canal (`ERQ0` a `ERQ15`): `1` = habilitado, `0` = deshabilitado. Para tocar un solo canal sin hacer lectura-modificación-escritura sobre todo `ERQ`, existen:

Registro	Para qué
`DMA_SERQ`	**Set** Enable Request. Escribir el número de canal en `SERQ` activa su bit en `ERQ`. Con `SAER = 1` activa **todos** los canales.
`DMA_CERQ`	**Clear** Enable Request. Escribir el número de canal en `CERQ` desactiva su bit. Con `CAER = 1` desactiva **todos**.

(En ambos, `NOP = 1` hace que el registro ignore la escritura.)

```
DMA0->SERQ = DMA_SERQ_SERQ(0);   // habilita las solicitudes del canal 0
DMA0->CERQ = DMA_CERQ_CERQ(0);   // deshabilita las solicitudes del canal 0
```

> ⚠️ **Orden de configuración**
>
> Primero se deja **listo el TCD y el DMAMUX**, y **recién después** se habilita el request en `ERQ`.

10. Buffer circular por hardware: `SMOD` y `DMOD`

Los campos `SMOD` (origen) y `DMOD` (destino) de `ATTR` convierten una región de memoria en un **buffer circular por hardware** de tamaño $M = 2^K$ bytes, con K = valor del campo. El hardware deja que cambien **solo los K bits menos significativos** de la dirección; el resto queda fijo, así que la dirección **da la vuelta** dentro del bloque.

- `K = 0` → no hay módulo (la dirección avanza normalmente).
- `K ≠ 0` → buffer circular de 2^K bytes.

> 💡 **Requisito: buffer alineado**
>
> Para que funcione, la dirección base del buffer tiene que ser **múltiplo de 2^K**: sus K bits menos significativos deben ser **cero**. Por ejemplo, con `SMOD = 2`: $K = 2$, $M = 2^2 = 4$ bytes, y `SADDR` debe terminar en `00` en binario.
>
> El módulo siempre es **potencia de dos**. Para un tamaño arbitrario se usa Scatter-Gather (más adelante).

🖼️ Buffer circular con `SMOD = 2` (M = 4): los 2 bits menos significativos de SADDR cambian y el resto queda fijo (diapositiva 33)

```
// Buffer circular de 4 bytes en el origen: SMOD = 2  →  M = 2^2 = 4
DMA0->TCD[0].ATTR = DMA_ATTR_SSIZE(0) | DMA_ATTR_DSIZE(0) | DMA_ATTR_SMOD(2);
```

Cómo alinear una variable en C

Con GCC se usa el atributo `aligned`: la dirección de inicio de la variable es múltiplo de $L = 2^n$ bytes.

```
int x __attribute__ ((aligned (16))) = 0;     // n = 4, L = 16 bytes
```

```
uint16_t sourceBuffer[10] __attribute__ ((aligned (16)))
                          = {0x1234, 0x6789, 0x1122, 0x2233, 0x5588};
uint8_t  destinationBuffer[10] __attribute__ ((aligned (16)));
```

Ejemplo paso a paso: buffer circular de destino de 4 bytes

Se copian bytes de una tabla de `uint16_t` hacia un buffer de destino que "da la vuelta" cada 4 bytes (`DMOD = 2`).

```
uint16_t sourceBuffer[10] __attribute__ ((aligned (4)))
                          = {0x1234, 0x6789, 0x1122, 0x2233, 0x5588};
uint8_t  destinationBuffer[10] __attribute__ ((aligned (4)));

DMA0->TCD[0].SOFF = 0x01;   // origen: avanza 1 byte por transferencia
DMA0->TCD[0].DOFF = 0x01;   // destino: avanza 1 byte por transferencia

// Datos de 1 byte (8 bits) y destino con módulo 4 (DMOD = 2)
DMA0->TCD[0].ATTR = DMA_ATTR_SSIZE(0) | DMA_ATTR_DSIZE(0) | DMA_ATTR_DMOD(2);

// Bytes por request (2 transferencias de 1 byte)
DMA0->TCD[0].NBYTES_MLNO = 0x02;

// Major loop: 5 requests
DMA0->TCD[0].CITER_ELINKNO = DMA_CITER_ELINKNO_CITER(0x05);
DMA0->TCD[0].BITER_ELINKNO = DMA_BITER_ELINKNO_BITER(0x05);
```

Los bytes de origen (little endian) son `34 12 89 67 22 11 33 22 ...`. Cada request mueve 2 bytes. Con `DADDR` terminando en `...9C` (sus 2 bits bajos en `00`), el destino da la vuelta cada 4 bytes:

Request	Bytes escritos	`DADDR` después	`SADDR` después	`CITER`
Inicio	—	`0x1FFF009C`	`0x1FFF0000`	5
1	`0x34`, `0x12` en `dest[0..1]`	`0x1FFF009E`	`0x1FFF0002`	4
2	`0x89`, `0x67` en `dest[2..3]`	`0x1FFF009C` (**wrap**)	`0x1FFF0004`	3
3	`0x22`, `0x11` en `dest[0..1]` (pisa)	`0x1FFF009E`	`0x1FFF0006`	2
4	`0x33`, `0x22` en `dest[2..3]` (pisa)	`0x1FFF009C` (**wrap**)	`0x1FFF0008`	1

El destino nunca sale de los 4 bytes (`dest[0..3]`): cuando `DADDR` llegaría a `...A0`, vuelve a `...9C`.

Capturas del debugger para cada request: contenido de `destinationBuffer` y registros `TCD0_DADDR`, `TCD0_SADDR`, `TCD0_CITER` (diapositiva 41)

11. Ejemplo completo: PWM con duty variable desde una tabla (DMA + FTM)

Objetivo: generar una señal PWM cuyo **duty cycle cambia automáticamente** en cada período, tomando los valores de una tabla en memoria, sin que la CPU haga nada.

```
uint16_t sourceBuffer[] = {1000, 2000, 3000, 4000, 5000, 6000, 7000, 8000};
```

Cómo funciona: en cada período del PWM, el FTM genera un request de DMA; el DMA copia el siguiente valor de la tabla al registro `CnV` del canal (el que fija el duty) y se queda esperando el próximo request. Al terminar la tabla, `SLAST` la reinicia.



Configuración del DMAMUX y del TCD

```
// DMAMUX: canal 0, fuente FTM0 canal 0 (slot 20), modo normal
DMAMUX->CHCFG[0] |= DMAMUX_CHCFG_ENBL_MASK | DMAMUX_CHCFG_SOURCE(20);

// TCD0
DMA0->TCD[0].SADDR = (uint32_t)(sourceBuffer);                // tabla de duties
DMA0->TCD[0].DADDR = (uint32_t)(&(FTM0->CONTROLS[0].CnV));    // registro que fija el duty

DMA0->TCD[0].SOFF = 0x02;   // origen: avanza 2 bytes (uint16_t)
DMA0->TCD[0].DOFF = 0x00;   // destino: siempre el mismo registro

DMA0->TCD[0].ATTR = DMA_ATTR_SSIZE(1) | DMA_ATTR_DSIZE(1);    // datos de 16 bits
DMA0->TCD[0].NBYTES_MLNO = 0x02;                              // 2 bytes por request

// Major loop: tantos requests como elementos tenga la tabla
DMA0->TCD[0].CITER_ELINKNO = DMA_CITER_ELINKNO_CITER(sizeof(sourceBuffer) /
sizeof(sourceBuffer[0]));
DMA0->TCD[0].BITER_ELINKNO = DMA_BITER_ELINKNO_BITER(sizeof(sourceBuffer) /
sizeof(sourceBuffer[0]));

// Al terminar el major loop, SADDR vuelve al principio de la tabla
DMA0->TCD[0].SLAST = -sizeof(sourceBuffer);

// Scatter-Gather no se usa
DMA0->TCD[0].DLASTSGA = 0x00;

DMA0->SERQ = DMA_SERQ_SERQ(0);   // ¡habilitar las solicitudes del canal 0!
```

Configuración del FTM0

Para que el FTM genere un request de DMA hay que habilitar, en `FTMx_CnSC`, **tanto** el bit `DMA` como `CHnIE`:

[illegible]	[illegible]	Request de DMA	Interrupción del canal
0	0	No se genera.	No se genera.
0	1	No se genera.	Se genera si `CHnF = 1`.
1	0	No se genera.	No se genera.

DMA	CHnIE	Request de DMA	Interrupción del canal
1	**1**	**Se genera si `CHnF = 1`.**	No se genera.

Con `DMA = 1`, el flag `CHnF` lo **limpia la transferencia DMA**, no el software.

```
uint16_t PWM_modulus = 10000 - 1;
uint16_t PWM_duty    = 5000;

FTM_SetPrescaler(FTM0, FTM_PSC_x32);                              // prescaler ×32
FTM_SetInterruptMode(FTM0, FTM_CH_0, true);                       // CHnIE = 1
FTM_SetWorkingMode(FTM0, 0, FTM_mPulseWidthModulation);           // modo PWM (MSA/MSB)
FTM_SetPulseWidthModulationLogic(FTM0, 0, FTM_lAssertedHigh);     // polaridad (ELSA/ELSB)
FTM_SetModulus(FTM0, PWM_modulus);                                // módulo (período)
FTM_SetCounter(FTM0, 0, PWM_duty);                                // duty inicial
FTM_DmaMode(FTM0, FTM_CH_0, FTM_DMA_ON);                          // DMA = 1
FTM_StartClock(FTM0);                                             // arrancar el timer
```

Tabla de requests de DMA del FTM y registro `FTMx_CnSC` con los bits `CHIE` y `DMA` resaltados (diapositiva 44)

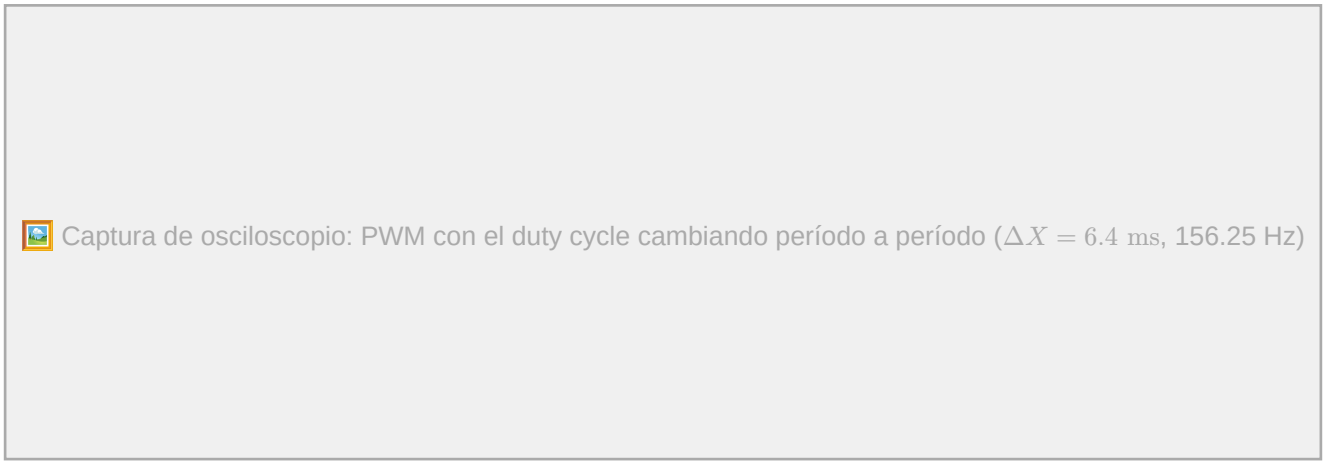
Resultado: cuentas

Con el prescaler en 32 y el reloj del FTM en 50 MHz, cada cuenta dura $\frac{32}{50\ \text{MHz}} = 0.64\mu\text{s}$.

$$T_{\text{PWM}} = \frac{1}{50 \times 10^6} \times 32 \times 10000 = 6.40\ \text{ms} \quad (\approx 156.25\ \text{Hz})$$

El ancho del pulso (duty) es $t_{\text{duty}} = 0.64\mu\text{s} \times \text{sourceBuffer[i]}$:

Índice	Valor en la tabla	Duty (tiempo en alto)
0	1000	640 µs
1	2000	1.28 ms
2	3000	1.92 ms
3	4000	2.56 ms
4	5000	3.20 ms
5	6000	3.84 ms
6	7000	4.48 ms
7	8000	5.12 ms



12. Modos avanzados: Scatter-Gather y Channel Linking

Ambos sirven para encadenar transferencias, pero de formas muy distintas.

Scatter-Gather: un canal, varios TCD

Scatter-Gather permite que **un mismo canal use varios TCD**. Cada vez que termina el major loop, el hardware **carga automáticamente** el siguiente TCD desde la memoria principal y lo copia al descriptor del canal, reemplazando al actual. Todo sin intervención de la CPU. El nombre viene de poder **dispersar** (scatter) datos hacia varios destinos o **juntar** (gather) datos desde varios orígenes.

- Se activa con `CSR[ESG] = 1` en los TCD.
- `DLASTSGA` de cada TCD contiene la **dirección del siguiente TCD**.
- Los TCD (`TCDA`, `TCDB`, …) viven en la **memoria principal** y deben estar **alineados a 32 bytes** (la región es modular de 32 bytes).
- Si el último apunta al primero, forman una **lista circular**.




```
/* Dos TCD en memoria principal, alineados a 32 bytes */
TCD_t tcd[TCDs_AMOUNT] __attribute__ ((aligned (32)));

enum { TCD0 = 0, TCD1, TCDs_AMOUNT };

tcd[TCD0].DLASTSGA = (uint32_t)&tcd[TCD1];   // TCD0 → el siguiente es TCD1
tcd[TCD1].DLASTSGA = (uint32_t)&tcd[TCD0];   // TCD1 → vuelve a TCD0 (circular)

DMA_TCD0_DLASTSGA = tcd[TCD0].DLASTSGA;      // el canal arranca apuntando al siguiente
```

Aplicaciones típicas: **buffers de tamaño arbitrario** (que no son potencia de 2) y esquemas de **doble buffer (ping-pong)**.

Channel-to-Channel Linking: un canal dispara a otro

El **Channel Linking** hace que, al completar un minor loop o un major loop, **se active otro canal que ya está configurado**. No se carga ningún TCD nuevo ni se cambia la configuración.

Cuándo se enlaza	Campo del TCD	Descripción
Al final del **minor loop**	`CITER[E_LINK]`	Habilita el linking al completar el minor loop.
	`CITER[LINKCH]`	Número del canal a activar.
Al final del **major loop**	`CSR[MAJOR_E_LINK]`	Habilita el linking al completar el major loop.
	`CSR[MAJOR_LINKCH]`	Número del canal a activar.

> ⚠️ **`E_LINK` en `CITER` y en `BITER`**
>
> Los bits `TCDn_CITER[E_LINK]` y `TCDn_BITER[E_LINK]` **deben ser iguales**; si no, se reporta un error de configuración. Además, el ancho de `CITER` y `BITER` debe ser el mismo para calcular el punto de interrupción de mitad del major loop.

Comparación

Característica	Scatter-Gather	Channel Linking
Canales involucrados	**Un solo canal** que va cambiando su configuración.	**Varios canales** independientes que se disparan entre sí.
¿Qué cambia?	Se reemplaza el TCD completo (origen, destino, tamaño), copiándolo desde la RAM.	No cambia ninguna configuración: solo se activa el trigger de otro canal ya configurado.
Uso de memoria	Requiere guardar estructuras TCD de 32 bytes en la SRAM.	No consume RAM adicional; se maneja con los registros de control del eDMA.

13. Ejemplo: buffer circular de 10 elementos (no potencia de 2)

`SMOD`/`DMOD` solo admiten tamaños potencia de dos, así que para un buffer de **10 elementos** hay que usar Scatter-Gather. El ejemplo está basado en la nota de aplicación *What is and how to configure the eDMA scatter-gather feature*.

Planteo:

- Hay 2 buffers circulares de 10 elementos: `src` (origen) y `dst` (destino).
- El DMA copia de `src` a `dst` en **2 etapas**, cada una un major loop de 5 elementos.
- **TCD A** copia los primeros 5 (`SADDR = src`, `DADDR = dst`, `CITER = 5`). **TCD B** copia los 5 restantes (`SADDR = src + 5`, `DADDR = dst + 5`, `CITER = 5`). Cada uno apunta al otro con `DLASTSGA`.
- Al terminar cada major loop se produce una **interrupción**: la CPU ingresa 5 elementos nuevos en `src` (para simular una nueva adquisición de datos) mientras el DMA continúa con la otra mitad.

Major loop	TCD	`src` después	`dst` después
Inicio	A	`0 1 2 3 4 5 6 7 8 9`	`0 0 0 0 0 0 0 0 0 0`
1	A	`A B C D E 5 6 7 8 9` (nuevos A a E)	`0 1 2 3 4 0 0 0 0 0`
2	B	`A B C D E F 10 11 12 13` (nuevos F a 13)	`0 1 2 3 4 5 6 7 8 9`
3	A	`14 15 16 17 18 F 10 11 12 13` (nuevos 14 a 18)	`A B C D E 5 6 7 8 9`

Major loop	TCD	[illegible] después	[illegible] después
4	B	(sigue igual)	`A B C D E F 10 11 12 13`

Los valores nuevos son números consecutivos escritos en hexadecimal (`A` = 10, `F` = 15, `10` = 16, etc.). Después del major loop 4 se vuelve a cargar TCD A y el ciclo continúa.

🖼️ Simulación: `src` y `dst` en cada major loop, alternando TCD A y TCD B, con los datos nuevos que carga la CPU en la interrupción

14. Herramienta de depuración: EmbSysRegView

Para **depurar y aprender** el DMA es muy útil poder ver los registros del eDMA (TCD, `ERQ`, etc.) mientras el programa corre. **EmbSysRegView** (*Embedded Systems Register View*) es una vista de Eclipse que muestra todos los registros del micro (en este caso de la familia MK64F12), con sus valores en hexa y binario, su dirección y una descripción de cada campo.

15. Resumen y checklist de configuración

💡 Receta para configurar un canal DMA

1. **Clocks:** habilitar `SIM_SCGC6[DMAMUX]` (y `SIM_SCGC7[DMA]`).
2. **Periférico:** configurarlo para que genere el request de DMA (por ejemplo, `IRQC` en un pin, o `DMA` y `CHnIE` en el FTM).
3. **DMAMUX:** configurar `CHCFGn` con `SOURCE`, `ENBL` y, si hace falta, `TRIG` (canales 0 a 3 con el PIT).
4. **TCD:** cargar `SADDR`, `DADDR`, `SOFF`, `DOFF`, `ATTR`, `NBYTES`, `CITER` = `BITER`, `SLAST`, `DLASTSGA` y `CSR`.
5. **PIT** (si se usa trigger periódico): configurar `LDVAL` y habilitar el timer (`TEN`).
6. **Habilitar el canal:** `DMA0->SERQ = DMA_SERQ_SERQ(n)`.

⚠️ Errores típicos

- Olvidarse de habilitar el clock del **DMAMUX** (arranca apagado).
- Olvidarse de habilitar el request del canal en `ERQ` (`SERQ`).
- `CITER` distinto de `BITER` al inicializar el TCD.
- `NBYTES = 0` (se interpreta como 4 GB).
- No poner `SLAST` negativo cuando se quiere repetir una tabla.
- Buffer circular (`SMOD`/`DMOD`) **sin alinear** a su tamaño.
- Usar trigger periódico (`TRIG = 1`) en un canal **mayor a 3**.
- Confundir **minor loop** (lo de un request) con **major loop** (todos los requests).

🛠️ Cadena completa por hardware

PIT (marca el ritmo) → **trigger** → **DMAMUX** (rutea) → **eDMA** (transfiere según el TCD) → **memoria o periférico**. La CPU solo configura al principio y atiende una interrupción al final de cada major loop (si se habilitó `INTMAJOR`).

Referencias

- *K64 Sub-Family Reference Manual*, Rev. 2, January 2014.
- *Using the DMA module in Kinetis devices*, Freescale Semiconductor.
- *What is and how to configure the eDMA scatter-gather feature* (nota de aplicación).
- *How to Add Register Details View in Eclipse (The EmbSysRegView project)*, https://mcuoneclipse.com/2014/05/29/how-to-add-register-details-view-in-eclipse/

Clase 10

Introducción a ARM Cortex-M4: Startup y Toolchain

> 💡 **Objetivo de la clase**
>
> El microcontrolador **no arranca directamente en `main()`**. Antes hay que dejar listo el **stack**, la **vector table** y la memoria (`.data` y `.bss`). En esta clase vemos (1) qué define la arquitectura ARM y cómo se organiza el Cortex-M4, (2) dónde vive cada variable en memoria, (3) qué hace la CPU después del reset y (4) cómo la **toolchain** genera el binario que se graba en la Flash.

CPU vs. microcontrolador

- El **ARM Cortex-M4** es una **CPU** (un núcleo).
- El **Kinetis MK64FN1M0VLL12** (NXP) es un **microcontrolador (MCU)**: la CPU más Flash, RAM, periféricos, reloj, lógica de reset, etc.

Esta distinción importa más adelante: **el reset lo maneja el MCU, no la CPU**.

Qué es ARM y qué define una arquitectura

- **ARM** (Advanced RISC Machine) es una **arquitectura de CPU**, no un fabricante.
- **Arm Holdings** es la empresa que **licencia** esa arquitectura a otras (NXP, ST, Microchip, Apple, etc.).
- La arquitectura es el **"contrato" entre hardware y software**: describe **qué** funcionalidad debe proveer el hardware al software, pero **no cómo** la implementa.

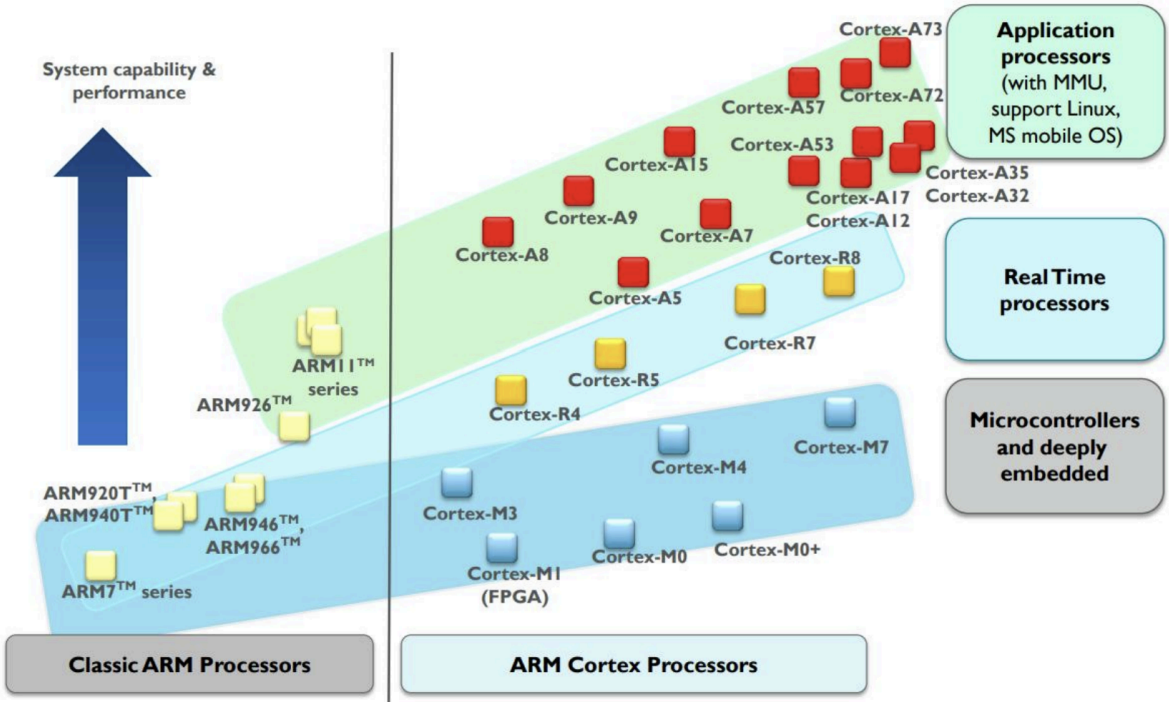
Elemento	Qué define
Instruction set (ISA)	Instrucciones disponibles, función de cada una (y qué excepciones puede disparar) y cómo se codifica en memoria.
Register set	Registros disponibles, su tamaño, su función y su comportamiento en el reset.
Exception model	Niveles de privilegio, tipos de excepciones y qué las dispara, y qué pasa al entrar o volver de una excepción.
Memory model	Orden de los accesos a memoria, comportamiento de las cachés y cuándo el software debe hacer mantenimiento explícito.
Debug, trace y profiling	Cómo se ponen y disparan los breakpoints, y qué información capturan las herramientas de trace y en qué formato.

Familias de procesadores ARM

ARM ofrece tres grandes perfiles, según el mercado al que apuntan:

	Cortex-A (Application)	**Cortex-R** (Real-time)	**Cortex-M** (Microcontroller)
Diseño	Alta frecuencia, pipeline largo, alta performance, soporte multimedia (NEON).	Pipeline corto a medio, determinístico (baja latencia de interrupción).	Pipeline corto, ultra bajo consumo, determinístico.
Features	MMU, caché, seguridad TrustZone, corre Linux/OS móviles.	MPU, caché, memoria TCM (Tightly Coupled Memory).	MPU, **NVIC** (controlador de interrupciones), WIC (wakeup).

	Cortex-A (Application)	**Cortex-R** (Real-time)	**Cortex-M** (Microcontroller)
Mercado	Smartphones, servidores, microprocesadores de alta gama.	Microcontroladores industriales, automotriz, discos rígidos, módems.	Sensores, MEMS, IoT, sistemas embebidos de bajo consumo.



Dentro de la familia Cortex-M:

Núcleo	**Característica principal**
M0 / M0+	Muy chicos, bajo costo y ultra bajo consumo. El M0+ agrega I/O de ciclo único y relocación de vector table.
M1	Diseñado para FPGAs (implementación con bloques de memoria de la FPGA). Mismo set de instrucciones que el M0.
M3	Más potente: set de instrucciones rico, divisor por hardware, MAC, debug y trace completos.
M4	Lo del M3 **más instrucciones DSP** (SIMD, MAC de ciclo único) y una **FPU opcional** de precisión simple (IEEE 754).
M7	Alto rendimiento: FPU de doble precisión opcional, cachés y TCM.

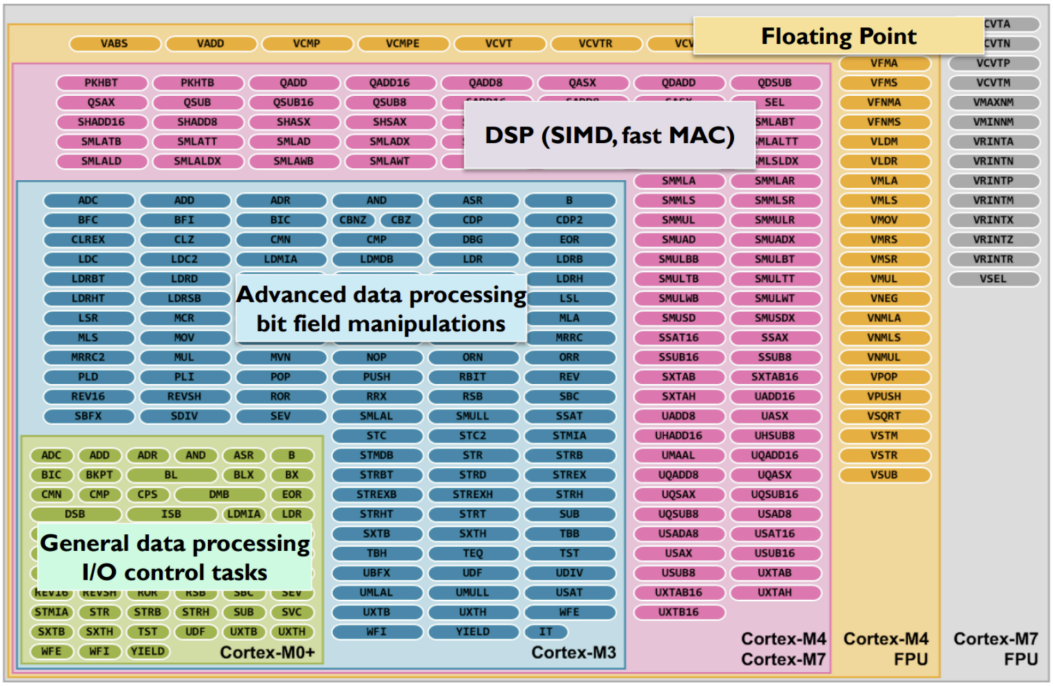


Figura 40: Soporte del Instruction Set en los distintos Cortex-M: M0/M0+, M3, M4/M7, DSP, FPU

Mapa de registros del Cortex-M4

El Cortex-M tiene registros de **32 bits**:

- **16 registros de propósito general** (`R0` a `R15`; tres de ellos tienen función especial: `SP`, `LR` y `PC`).
- **5 registros especiales**.
- **32 registros de punto flotante** (`S0`–`S31`, solo en variantes con FPU).

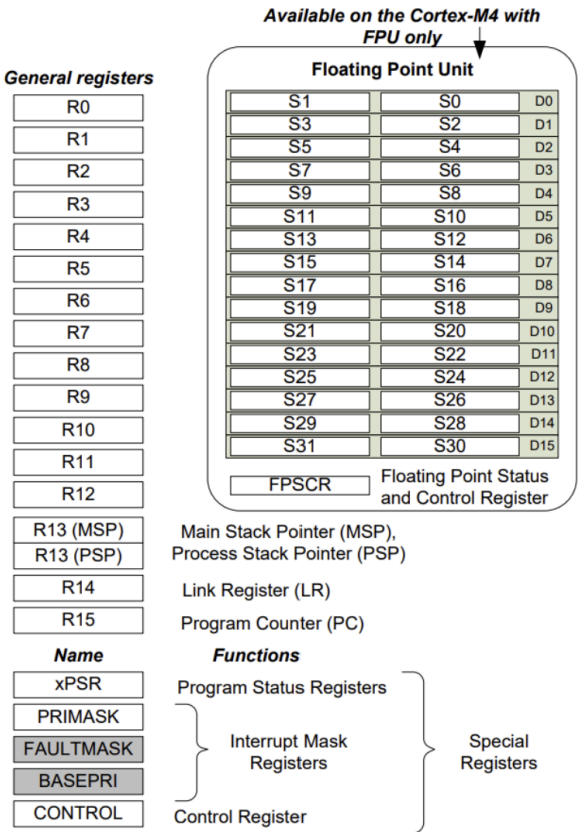


Figura 41: Mapa de registros del Cortex-M (R0–R15, registros especiales y registros de FPU

Registro	Función
`R0–R12`	Propósito general.
`R13` (`SP`)	**Stack Pointer**: puntero al stack. Existen dos copias, `MSP` (Main) y `PSP` (Process).
`R14` (`LR`)	**Link Register**: dirección de retorno de subrutina.
`R15` (`PC`)	**Program Counter**: instrucción que se está ejecutando.
`PSR`	Flags de la ALU, información de interrupciones y estado de ejecución.
`PRIMASK` (1 bit)	Bloquea **todas** las excepciones (incluidas las interrupciones), excepto `NMI` y `HardFault`.
`FAULTMASK` (1 bit)	Bloquea todas las excepciones, **incluso** `HardFault` (queda solo la `NMI`).
`BASEPRI` (8 bits)	Bloquea las excepciones con prioridad "mayor o igual a su valor" (numéricamente: valor de prioridad ≥ `BASEPRI`, o sea las menos urgentes).
`CONTROL`	Selecciona qué stack pointer se usa (`MSP` o `PSP`) y otras opciones de control.

> ⚠️ **Prioridades en ARM**
>
> En Cortex-M, **número menor = prioridad más alta**. Por eso `BASEPRI` enmascara las excepciones cuyo número de prioridad es **igual o mayor** al configurado (las menos urgentes) y deja pasar las más urgentes.

El Stack Pointer en acción

El stack se usa para guardar temporalmente registros y direcciones de retorno. Ejemplo típico: una subrutina que preserva `R4` y `R5` del llamador.

```
        ; Programa principal
        ; R4 = A, R5 = B
        BL   func1          ; salta a func1 y guarda en LR la dirección de retorno
        ; al volver: R4 = A, R5 = B

func1:
        PUSH {R4-R5}        ; guardar R4 y R5 en el stack
        ; ... tareas de la subrutina (puede usar R4 y R5 libremente) ...
        POP  {R4-R5}        ; restaurar R4 y R5
        BX   LR             ; retornar a main
```

- `PUSH` **decrementa** `SP` y escribe los registros: el stack **crece hacia direcciones bajas**.
- `POP` lee los valores y **incrementa** `SP`.

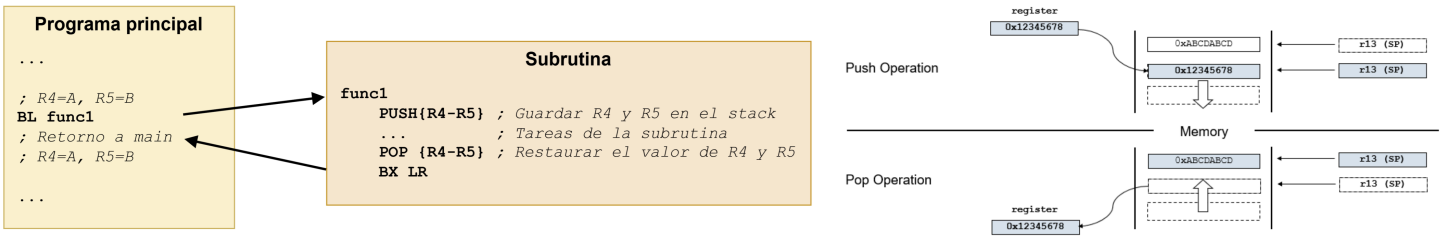


Figura 42: Operaciones Push y Pop sobre el stack

Arquitectura del Cortex-M4 y mapa de memoria

La arquitectura del Cortex-M4 es un **híbrido entre Harvard y Von Neumann**:

- Tiene **buses separados** para código y datos (como Harvard).

- Pero tiene un **único mapa de memoria unificado** (como Von Neumann).

Además, tiene componentes opcionales (FPU, MPU, ETM, watchpoints), el **NVIC** (que unifica el controlador de interrupciones entre los distintos fabricantes) y el **Debug Access Port**.

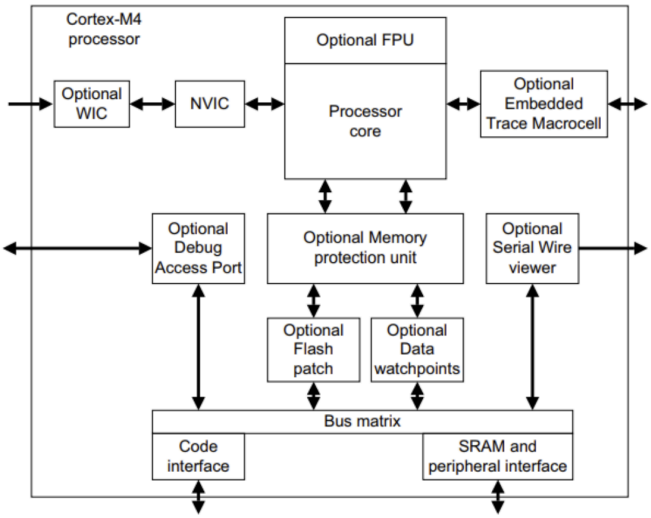


Figura 43: Diagrama de bloques de la implementación del Cortex-M4

> 💡 **Todos los Cortex-M4 comparten el mapa de memoria**
>
> El **mapa de memoria** (qué rango de direcciones corresponde a código, SRAM, periféricos, etc.) es **el mismo para todas las implementaciones** del Cortex-M4. Lo que **sí depende del fabricante/producto** es **qué hay realmente** en cada región: cuánta Flash y RAM tiene, qué periféricos incluye y en qué direcciones están sus registros.

Región	Tamaño	Rango de direcciones
Code (Flash)	0.5 GB	`0x0000_0000` – `0x1FFF_FFFF`
SRAM	0.5 GB	`0x2000_0000` – `0x3FFF_FFFF`
Peripheral	0.5 GB	`0x4000_0000` – `0x5FFF_FFFF`
External RAM	1 GB	`0x6000_0000` – `0x9FFF_FFFF`
External device	1 GB	`0xA000_0000` – `0xDFFF_FFFF`
Private peripheral bus	—	`0xE000_0000` – `0xE00F_FFFF`
System	—	`0xE010_0000` – `0xFFFF_FFFF`

Las regiones SRAM y Peripheral incluyen una **bit-band region** de 1 MB (con su **bit-band alias** de 32 MB) que permite acceder a bits individuales con una sola operación de lectura/escritura.

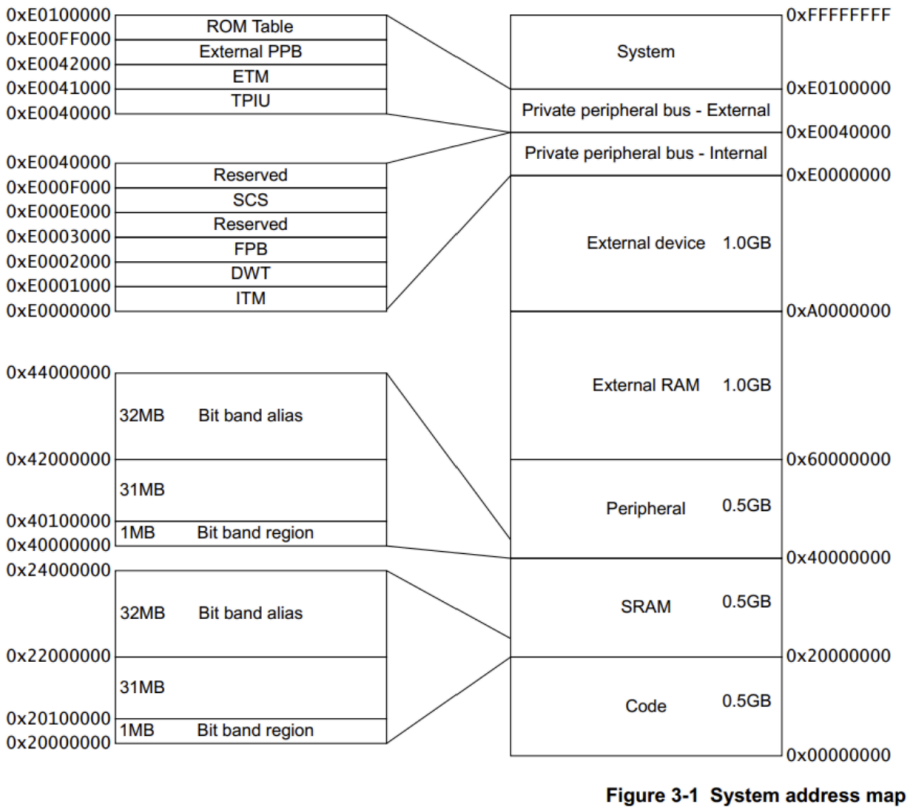


Figura 44: Mapa de memoria del sistema del Cortex-M4

¿Dónde se guarda cada variable?

Al compilar, el programa se divide en **secciones**. Cada tipo de variable o función cae en una sección distinta, y cada sección va a un tipo de memoria distinto.

Sección	Memoria	Qué contiene
`.text`	Flash	Código (instrucciones).
`.rodata`	Flash	Constantes (read-only).
`.data`	RAM (valores iniciales en Flash)	Variables globales/estáticas inicializadas con valor distinto de 0.
`.bss`	RAM	Variables globales/estáticas inicializadas en 0 (o no inicializadas).
heap	RAM	Memoria reservada dinámicamente (`malloc`).
stack	RAM	Variables locales y contexto de las llamadas a función.

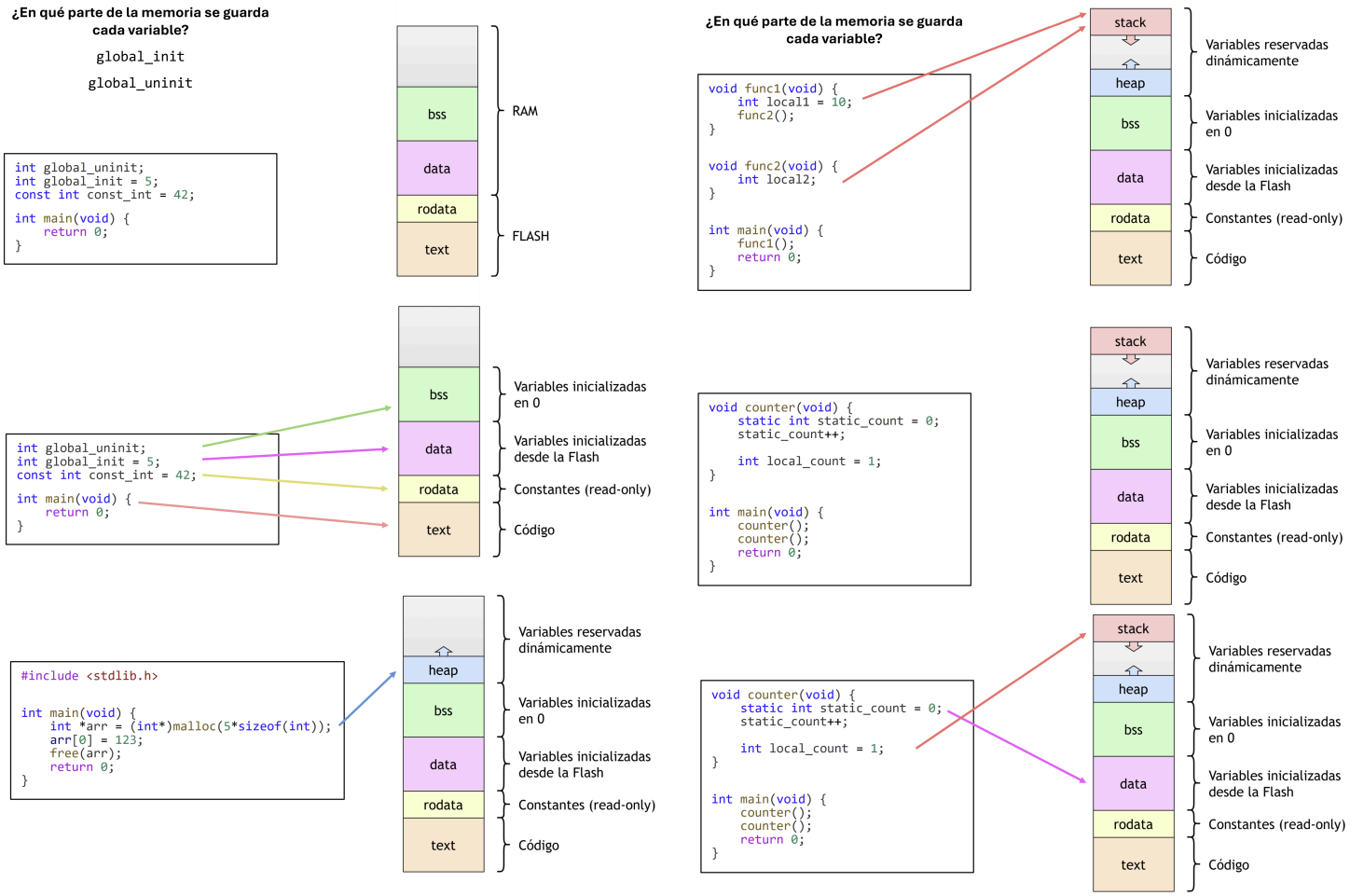


Figura 45: Layout de memoria: text, rodata, data, bss, heap y stack

Ejemplos de la clase

```c
const int const_int = 42;
const char * const const_string = "Hola";
int global_uninit;
int global_init = 5;

int main(void) {
    return 0;
}
```

Símbolo	Sección	Por qué
`main()`	`.text`	Es código.
`const_int`	`.rodata`	Constante: solo lectura, queda en Flash.
`const_string`	`.rodata`	Puntero constante a una cadena constante; ambos en Flash.
`global_init`	`.data`	Inicializada en 5: vive en RAM, pero su valor inicial se guarda en Flash.
`global_uninit`	`.bss`	Sin inicializar: vive en RAM y debe ponerse en 0.

Memoria dinámica (heap):

```c
#include <stdlib.h>

int main(void) {
    int *arr = (int*)malloc(5*sizeof(int));
    arr[0] = 123;
    free(arr);
    return 0;
}
```

- El bloque de 5 enteros que devuelve `malloc` está en el **heap**.
- El puntero `arr` en sí es una variable local y está en el **stack**.

Variables locales (stack):

```c
void func2(void) {
    int local2;
}

void func1(void) {
    int local1 = 10;
    func2();
}

int main(void) {
    func1();
    return 0;
}
```

`local1` y `local2` están en el **stack**. Cada llamada a función apila un nuevo marco, por eso `local2` queda "encima" de `local1` y desaparece al retornar `func2`.

Variables `static` locales:

```c
void counter(void) {
    static int static_count = 0;
    static_count++;

    int local_count = 1;
}

int main(void) {
    counter();
    counter();
    return 0;
}
```

- `local_count` está en el **stack**: se crea y se destruye en cada llamada.
- `static_count` **no está en el stack**: es una variable global con alcance local a la función, por eso **conserva su valor entre llamadas** (después de dos llamadas vale 2).

> ⚠️ **`static` inicializada en 0**
>
> En la diapositiva `static_count` se muestra en `.data`. En la práctica, GCC suele ubicar en `.bss` las variables inicializadas explícitamente en 0. En cualquier caso, lo importante es que **vive en RAM, fuera del stack**.

Booteo del ARM Cortex-M4

Supongamos el programa mínimo:

```c
void main(void) {
    while (1) {
    }
}
```

La pregunta es: **¿cuál es la secuencia exacta desde que se energiza el micro hasta que ejecuta la primera instrucción de `main`? ¿Hay algo antes?**

Del power-on al reset

Al energizar el micro, la tensión empieza a subir. Mientras tanto, el MCU mantiene la CPU en reset. **El reset lo maneja la implementación del fabricante (el MCU), no la CPU Cortex-M4.**

- En el K64, el **Power-On Reset (POR)** se dispara cuando se aplica alimentación (o cuando cae por debajo de cierto umbral), y el circuito **LVD** (Low-Voltage Detect) mantiene el MCU en reset hasta que la tensión supera el umbral.
- Los valores concretos (tensión de POR, umbral de LVD) están en el **K64 Sub-Family Reference Manual** (sección Power-On Reset) y en el **Kinetis K64F Data Sheet** (requisitos de LVD y POR).

Qué hace la CPU después del reset

Después del reset (Power-On Reset u otro), la CPU:

1. Carga en `MSP` el **valor guardado en la dirección** `0x0000_0000`.
2. Carga en `LR` el valor `0xFFFF_FFFF`.
3. Carga en `PC` el **valor guardado en la dirección** `0x0000_0004`.

> ⚠️ **Es el contenido, no la dirección**
>
> `MSP` y `PC` se cargan con **lo que hay almacenado** en las direcciones 0 y 4, no con los números 0 y 4. La primera palabra de la memoria es el **valor inicial del stack pointer** y la segunda es la **dirección del handler de reset**.

Para bootear un Cortex-M4 hay que poner entonces en la Flash:

- Un valor en la dirección `0x0` para cargar en `MSP` (¿cuál?).
- Un valor en la dirección `0x4` para cargar en `PC` (¿cuál?).
- Los valores iniciales de las variables globales y las constantes.
- ¡Un programa!

Y además, **antes de ejecutar `main`**, el programa debe:

- cargar el valor **0** en las variables no inicializadas (`.bss`);
- **copiar de Flash a RAM** los valores iniciales de las variables inicializadas (`.data`).

> 💡 **Secuencia de arranque**
>
> **Reset** (hardware del MCU) → la CPU carga `MSP` y `PC` desde la **vector table** → se ejecuta el **reset handler** (startup) → se inicializan `.bss` y `.data` → se llama a `main()` → bucle infinito.

La toolchain ARM

Queremos generar, sin IDE ni templates (solo un editor de texto), el **contenido binario que va en la Flash**. Eso lo hace la **toolchain**.

ARM Toolchain: Flujo genérico

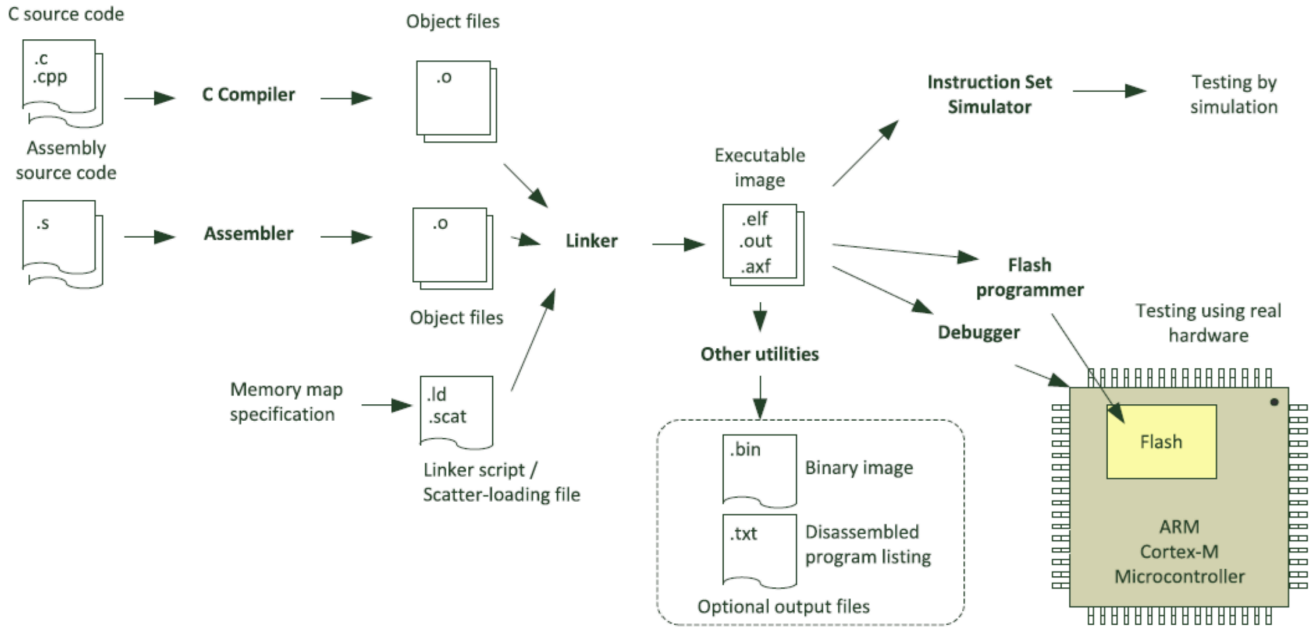


Figura 46: Flujo genérico de la toolchain ARM: compilador, ensamblador, linker, objcopy y flash programmer

- El **compilador** (C) y el **ensamblador** (assembly) generan **archivos objeto** (`.o`) de los codigos fuentes(c o assmebler).
- El **linker** combina los archivos objeto, según un **linker script** (la especificación del mapa de memoria), y genera la **imagen ejecutable** (`.elf`).
- Utilidades como `objcopy` convierten el ELF a formatos como `.bin` o `.hex`, y `objdump` genera un listado desensamblado.
- El **flash programmer** o el **debugger** graba esa imagen en la Flash del MCU real.

Nosotros usamos la toolchain **GNU** para ARM (`arm-none-eabi-*`, donde `none-eabi` indica programación **bare-metal**, sin sistema operativo).

Herramienta	Rol
`arm-none-eabi-gcc`	Compila C y ensambla assembly; también invoca al linker (`ld`) para generar el ELF.
`arm-none-eabi-ld`	Linker: asigna direcciones finales a las secciones y genera el ELF.
`arm-none-eabi-objdump`	Muestra el desensamblado y las tablas de símbolos y secciones.
`arm-none-eabi-objcopy`	Convierte el ELF a binario o hex para grabar en Flash.

Compilar: el archivo objeto

```
$ arm-none-eabi-gcc -mcpu=cortex-m4 -O1 -c ej0.c -o ej0.o
$ arm-none-eabi-objdump -d ej0.o
```

```
ej0.o:     file format elf32-littlearm

Disassembly of section .text:

00000000 <main>:
   0:   e7fe            b.n    0 <main>
```

`b.n 0 <main>` es el `while (1)`: un salto de `main` a sí misma. La dirección es `0` porque un archivo objeto **todavía no fue linkeado**: las direcciones son relativas al inicio de su sección.

Linkear sin indicarle dónde va el código

```
$ arm-none-eabi-gcc -mcpu=cortex-m4 -nostartfiles ej0.o -o ej0.elf
warning: cannot find entry symbol _start; defaulting to 0000000000008000
$ arm-none-eabi-objdump -d ej0.elf
```

```
00008000 <main>:
    8000:       e7fe                b.n       8000 <main>
```

Ahora `main` quedó en `0x8000`: es la dirección por defecto del linker. **No es "incorrecto", pero no es lo que queremos**: el Cortex-M4 espera su vector table en la dirección 0 de la Flash.

> 💡 **¿A quién le decimos dónde poner el código?**
>
> **Al linker**, no al compilador. El compilador solo **asigna secciones** a cada pedazo del programa; el **linker asigna direcciones** a esas secciones, guiado por el **linker script** (`-T script.ld`).

Secciones y linker script

Programa de ejemplo:

```c
static int p = 100;
int q;
static const int r = 9;

void main(void) {
    while (p-- > r) {
        q++;
    }
}
```

```
$ arm-none-eabi-objdump -td ej1.elf
SYMBOL TABLE:
00008000 l  d .text      00000000 .text
00008030 l  d .rodata    00000000 .rodata
00018034 l  d .data      00000000 .data
00018038 l  d .bss       00000000 .bss
00018034 l  O .data      00000004 p
00008030 l  O .rodata    00000004 r
0001803c g    .bss       00000000 _bss_end__
00018038 g    .bss       00000000 __bss_start__
0001803c g    .bss       00000000 __bss_end__
00018038 g    .bss       00000000 __bss_start
00008000 g  F .text      00000030 main
0001803c g    .bss       00000000 __end__
00018038 g    .data      00000000 _edata
0001803c g    .bss       00000000 _end
00018038 g  O .bss       00000004 q
00018034 g    .data      00000000 __data_start
```

El compilador reparte cada elemento en su sección estándar:

Sección	Contenido	En el ejemplo
`.text`	Código.	`main`

Sección	Contenido	En el ejemplo
`.rodata`	Datos inicializados != 0, solo lectura (RO).	`r`
`.data`	Datos inicializados != 0, lectura/escritura (RW).	`p`
`.bss`	Datos inicializados en 0.	`q`

Sin linker script, el linker usa direcciones por defecto: `.text` en `0x8000`, `.rodata` en `0x8030`, y `.data` y `.bss` pegadas más arriba (`0x18034`, `0x18038`). **Ninguna de esas direcciones es útil para nuestro micro**, así que hace falta un script propio.

El linker script

```
MEMORY
{
    flash : ORIGIN = 0x00000000, LENGTH = 1024k
    ram   : ORIGIN = 0x20000000, LENGTH = 256K
}

SECTIONS
{
    .text :
    {
        . = ALIGN(4);
        *(.text*)
        *(.rodata*)
    } > flash

    .data :
    {
        . = ALIGN(4);
        _data_start = .;
        *(.data*)
        . = ALIGN(4);
        _data_end = .;
    } > ram AT > flash

    _data_init_values = LOADADDR(.data);

    .bss (NOLOAD) :
    {
        . = ALIGN(4);
        _bss_start = . ;
        *(.bss*)
        *(COMMON)
        . = ALIGN(4);
        _bss_end = . ;
    } > ram
}
```

- `MEMORY` define las regiones físicas: `flash` (código y constantes) y `ram` (datos, stack).
- `.text` junta código y `.rodata` en Flash.
- `.data` se coloca en `ram` (**VMA**: dirección de ejecución) pero su contenido inicial se guarda en `flash` (**LMA**: dirección de carga), por eso `> ram AT > flash`. `LOADADDR(.data)` devuelve dónde quedaron esos valores iniciales en Flash.
- `.bss` es `NOLOAD`: ocupa RAM pero **no ocupa espacio en la Flash** (solo hay que ponerla en 0).
- El linker **define símbolos** (`_data_start`, `_data_end`, `_data_init_values`, `_bss_start`, `_bss_end`) que el código de inicio puede usar.

> ⚙️ **Nota sobre la RAM del K64**
>
> En los ejemplos de la clase la RAM arranca en `0x2000_0000` por simplicidad. En el K64 real, el reference manual reparte la SRAM a ambos lados de esa dirección (una parte arranca en `0x1FFF_0000`). Para un proyecto real hay que usar los valores del manual del MCU.

Compilar con el script

```
$ arm-none-eabi-gcc -mcpu=cortex-m4 -nostartfiles -T ej1.ld ej1.c -o ej1.elf
$ arm-none-eabi-objdump -td ej1.elf
SYMBOL TABLE:
00000000 l    d .text     00000000 .text
20000000 l    d .data     00000000 .data
20000004 l    d .bss      00000000 .bss
20000000 l    O .data     00000004 p
00000030 l    O .text     00000004 r
20000004 g    O .bss      00000004 q
00000000 g    F .text     00000030 main
20000000 g      .data     00000000 _data_start
20000004 g      .data     00000000 _data_end
20000004 g      .bss      00000000 _bss_start
20000008 g      .bss      00000000 _bss_end
00000034 g      *ABS*     00000000 _data_init_values
```

Ahora `main` está en `0x0`, `r` quedó en Flash (`0x30`), `p` y `q` en RAM (`0x2000_0000` y `0x2000_0004`), y los valores iniciales de `.data` están en Flash a partir de `0x34`.

El binario que va a la Flash

```
$ arm-none-eabi-objcopy -O binary ej1.elf ej1.bin
```

```
Offset    0   1   2   3   4   5   6   7   8   9   A   B   C   D   E   F
00000000  80  B4  00  AF  04  E0  08  4B  1B  68  01  33  06  4A  13  60
00000010  06  4B  1B  68  5A  1E  05  49  0A  60  09  22  93  42  F2  DC
00000020  00  BF  BD  46  80  BC  70  47  04  00  00  20  00  00  00  20
00000030  09  00  00  00  64  00  00  00
```

- Desde `0x00` hasta `0x2F`: el código de `main`.
- `0x30`: `09 00 00 00` = `r` = 9 (little endian).
- `0x34`: `64 00 00 00` = valor inicial de `p` = 100 (`0x64`), que alguien tendrá que **copiar a RAM**.

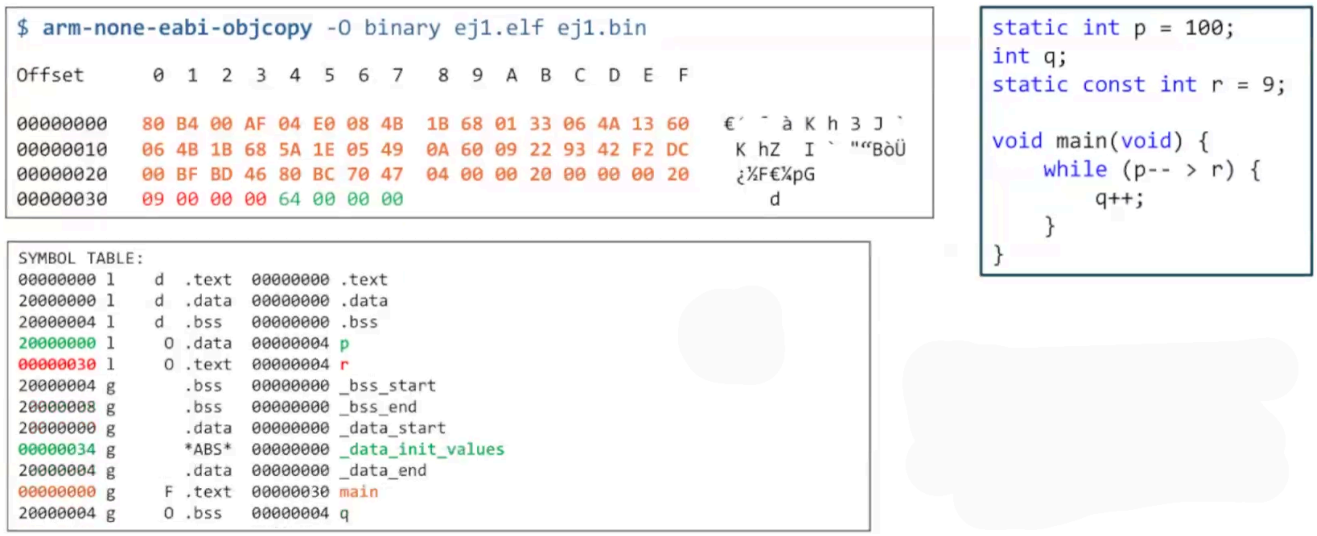


Figura 47: Ejemplo anterior.

> ⚠️ **¿Qué falta configurar?**
>
> Hasta acá el linker ordenó todo, pero **nadie usa los símbolos que generó**: no hay código que ponga `.bss` en 0 ni que copie `.data` a RAM. Y tampoco hay nada en las direcciones 0 y 4 que le diga al micro con qué `MSP` y `PC` arrancar. Falta el **código de startup**.

Código de startup

Inicialización pre-main (`init.c`)

```c
#include <string.h>

void init(void) {
    extern char _bss_start[];
    extern char _bss_end[];

    extern char _data_start[];
    extern char _data_end[];
    extern char _data_init_values[];

    // Poner .bss en 0
    memset(_bss_start, 0, (_bss_end - _bss_start));

    // Copiar los valores iniciales de .data desde Flash a RAM
    memcpy(_data_start, _data_init_values, (_data_end - _data_start));
}
```

Usa exactamente los símbolos que definió el linker script.

Vector table y reset handler (`startup.S`)

```
    .cpu cortex-m4
    .global _reset_vector

/* sección ".vectors", allocatable */
.section .vectors, "a"
    .word _sp_init_value      @ dirección 0x0: valor inicial de MSP
    .word _reset_vector       @ dirección 0x4: valor inicial de PC

.section .text
_reset_vector:
    bl init                   @ inicializar .bss y .data
    bl main                   @ ejecutar el programa de usuario
    b .                       @ si main retorna, quedarse acá
```

Las dos primeras palabras de la Flash son justo lo que la CPU lee después del reset: el stack inicial y la dirección del reset handler.

Linker script completo (`ej2.ld`)

```
MEMORY
{
    flash : org = 0, len = 1024k
    ram   : org = 0x20000000, len = 256K
}

ENTRY(_reset_vector)

SECTIONS
{
    .vectors :
```

```
    {
        *(.vectors*)
    } > flash

    ... (.text, .data y .bss como antes) ...

    _sp_init_value = org(ram) + len(ram);
}
```

- `.vectors` va **primero** en la Flash, para quedar en la dirección 0.
- `ENTRY(_reset_vector)` indica el punto de entrada.
- `_sp_init_value = org(ram) + len(ram)` coloca el stack al **final de la RAM**, y como el stack crece hacia abajo, tiene toda la RAM disponible.

Compilar y linkear todo junto

```
$ arm-none-eabi-gcc -mcpu=cortex-m4 -O0 -c startup.S -o startup.o
$ arm-none-eabi-gcc -mcpu=cortex-m4 -O0 -c init.c -o init.o
$ arm-none-eabi-gcc -mcpu=cortex-m4 -O0 -c ej2.c -o ej2.o
$ arm-none-eabi-gcc -mcpu=cortex-m4 -nostartfiles -T ej2.ld startup.o init.o ej2.o -o ej2.elf
$ arm-none-eabi-objdump -td ej2.elf > ej2.dump.txt
$ arm-none-eabi-objcopy -O binary ej2.elf ej2.bin
```

Al inspeccionar el binario, el primer dato es `00 00 04 20`, que en little endian es `0x2004_0000`: el valor inicial de `MSP`, es decir **`0x2000_0000` + 256 KB** (el tope de la RAM). La segunda palabra es la dirección del reset handler, y a continuación está el código de `startup`, `init` y `main`.

> **🛠️ Flujo completo de nuestro ejemplo**
>
> 1. Reset del MCU.
> 2. La CPU lee la dirección `0x0` y carga `MSP = 0x2004_0000`.
> 3. La CPU lee la dirección `0x4` y carga `PC = _reset_vector`.
> 4. `_reset_vector` llama a `init` (`.bss` en 0, `.data` copiada de Flash a RAM).
> 5. Se llama a `main`.
> 6. Si `main` retorna, `b .` deja la CPU en un bucle infinito.

Startup real del Kinetis (`startup_mk64f12.c`)

El SDK de Kinetis hace lo mismo pero en C y con la **vector table completa**.

Vector table. Es un arreglo de punteros a función que el linker script ubica al inicio de la Flash:

Dirección	Vector	Contenido
`0x0000_0000`	0	Valor inicial del stack pointer.
`0x0000_0004`	1	`ResetISR` (reset handler).
`0x0000_0008`	2	`NMI_Handler`.
`0x0000_000C`	3	`HardFault_Handler`.
`0x0000_0010`	4	`MemManage_Handler` (MPU fault).
`0x0000_0014`	5	`BusFault_Handler`.
`0x0000_0018`	6	`UsageFault_Handler`.
`0x0000_002C`	11	`SVC_Handler` (supervisor call).
`0x0000_0030`	12	`DebugMon_Handler`.

Dirección	Vector	Contenido
`0x0000_0038`	14	`PendSV_Handler`.
`0x0000_003C`	15	`SysTick_Handler`.
`0x0000_0040`	16 (IRQ 0)	Primera interrupción de periférico del chip (DMA canal 0).

Las posiciones reservadas se rellenan con `0`. Los primeros 16 vectores son **del núcleo ARM**; desde el vector 16 en adelante son **específicos del chip** (los periféricos del K64).

```
//*****************************************************************************
// The vector table.
// This relies on the linker script to place at correct location in memory.
//*****************************************************************************
extern void (* const g_pfnVectors[])(void);
extern void * __Vectors __attribute__ ((alias ("g_pfnVectors")));

__attribute__ ((used, section(".isr_vector")))
void (* const g_pfnVectors[])(void) = {
    // Core Level - CM4
    &_vStackTop,                    // The initial stack pointer
    ResetISR,                       // The reset handler
    NMI_Handler,                    // The NMI handler
    HardFault_Handler,              // The hard fault handler
    MemManage_Handler,              // The MPU fault handler
    BusFault_Handler,               // The bus fault handler
    UsageFault_Handler,             // The usage fault handler
    0,                              // Reserved
    0,                              // Reserved
    0,                              // Reserved
    0,                              // Reserved
    SVC_Handler,                    // SVCall handler
    DebugMon_Handler,               // Debug monitor handler
    0,                              // Reserved
    PendSV_Handler,                 // The PendSV handler
    SysTick_Handler,                // The SysTick handler

    // Chip Level - MK64F12
    DMA0_IRQHandler,                    // 16 : DMA Channel 0 Transfer Complete
    ...
```

Table 3-5. Interrupt vector assignments

Address	Vector	IRQ[1]	NVIC non-IPR register number [2]	NVIC IPR register number [3]	Source module	Source description
ARM Core System Handler Vectors						
0x0000_0000	0	–	–	–	ARM core	Initial Stack Pointer
0x0000_0004	1	–	–	–	ARM core	Initial Program Counter
0x0000_0008	2	–	–	–	ARM core	Non-maskable Interrupt (NMI)
0x0000_000C	3	–	–	–	ARM core	Hard Fault
0x0000_0010	4	–	–	–	ARM core	MemManage Fault
0x0000_0014	5	–	–	–	ARM core	Bus Fault
0x0000_0018	6	–	–	–	ARM core	Usage Fault
0x0000_001C	7	–	–	–	—	—
0x0000_0020	8	–	–	–	—	—
0x0000_0024	9	–	–	–	—	—
0x0000_0028	10	–	–	–	—	—
0x0000_002C	11	–	–	–	ARM core	Supervisor call (SVCall)
0x0000_0030	12	–	–	–	ARM core	Debug Monitor
0x0000_0034	13	–	–	–	—	—
0x0000_0038	14	–	–	–	ARM core	Pendable request for system service (PendableSrvReq)
0x0000_003C	15	–	–	–	ARM core	System tick timer (SysTick)
Non-Core Vectors						
0x0000_0040	16	0	0	0	DMA	DMA channel 0 transfer complete

Figura 48: Vector table del K64 (`g_pfnVectors`) y tabla de asignación de vectores de interrupción

Inicialización de RAM. Las funciones `data_init` y `bss_init` hacen lo mismo que nuestro `init.c`, pero copian/limpian palabra por palabra (de a 4 bytes):

```c
__attribute__ ((section(".after_vectors.init_data")))
void data_init(unsigned int romstart, unsigned int start, unsigned int len) {
    unsigned int *pulDest = (unsigned int*) start;
    unsigned int *pulSrc  = (unsigned int*) romstart;
    unsigned int loop;
    for (loop = 0; loop < len; loop = loop + 4)
        *pulDest++ = *pulSrc++;
}

__attribute__ ((section(".after_vectors.init_bss")))
void bss_init(unsigned int start, unsigned int len) {
    unsigned int *pulDest = (unsigned int*) start;
    unsigned int loop;
    for (loop = 0; loop < len; loop = loop + 4)
        *pulDest++ = 0;
}
```

Reset handler. `ResetISR` recorre unas **tablas de secciones** (`__data_section_table` y `__bss_section_table`) para inicializar cada banco de RAM, y finalmente llama a `main()`. Se usan tablas porque algunos MCUs tienen **varias regiones de RAM**, y así el mismo `ResetISR` sirve para todos. Si `main` retorna, queda en un `while (1)`.

```c
//*****************************************************************************
// Functions to carry out the initialization of RW and BSS data sections. These
// are written as separate functions rather than being inlined within the
// ResetISR() function in order to cope with MCUs with multiple banks of
// memory.
//*****************************************************************************
__attribute__ ((section(".after_vectors.init_data")))
void data_init(unsigned int romstart, unsigned int start, unsigned int len) {
    unsigned int *pulDest = (unsigned int*) start;
    unsigned int *pulSrc = (unsigned int*) romstart;
    unsigned int loop;
    for (loop = 0; loop < len; loop = loop + 4)
        *pulDest++ = *pulSrc++;
}

__attribute__ ((section(".after_vectors.init_bss")))
void bss_init(unsigned int start, unsigned int len) {
    unsigned int *pulDest = (unsigned int*) start;
    unsigned int loop;
    for (loop = 0; loop < len; loop = loop + 4)
        *pulDest++ = 0;
}
```

```c
//*****************************************************************************
// Reset entry point for your code.
// Sets up a simple runtime environment and initializes the C/C++
// library.
//*****************************************************************************
__attribute__ ((section(".after_vectors.reset")))
void ResetISR(void) {
    ...

    //
    // Copy the data sections from flash to SRAM.
    //
    unsigned int LoadAddr, ExeAddr, SectionLen;
    unsigned int *SectionTableAddr;

    // Load base address of Global Section Table
    SectionTableAddr = &__data_section_table;

    // Copy the data sections from flash to SRAM.
    while (SectionTableAddr < &__data_section_table_end) {
        LoadAddr = *SectionTableAddr++;
        ExeAddr = *SectionTableAddr++;
        SectionLen = *SectionTableAddr++;
        data_init(LoadAddr, ExeAddr, SectionLen);
    }
```

```c
    // At this point, SectionTableAddr = &__bss_section_table;
    // Zero fill the bss segment
    while (SectionTableAddr < &__bss_section_table_end) {
        ExeAddr = *SectionTableAddr++;
        SectionLen = *SectionTableAddr++;
        bss_init(ExeAddr, SectionLen);
    }

    ...

    main();

    //
    // main() shouldn't return, but if it does, we'll just enter an infinite loop
    //
    while (1) {
        ;
    }
}
```

Figura 49: `ResetISR`: copia de `.data` desde Flash a SRAM, puesta en cero de `.bss` y salto a `main`

Resumen final

💡 Ideas clave

1. **ARM define una arquitectura** (ISA, registros, excepciones, memoria, debug); el Cortex-M4 es una CPU, y el MCU (ej. Kinetis K64) es lo que la rodea.
2. **Todos los Cortex-M4 tienen el mismo mapa de memoria**; lo que cambia entre fabricantes es qué hay en cada región.
3. Cada variable va a una sección según su tipo: `.text`, `.rodata`, `.data`, `.bss`, **heap** y **stack**.
4. Tras el reset, la CPU carga `MSP` con el contenido de la dirección `0x0` y `PC` con el contenido de la dirección `0x4`: esa es la **vector table**.
5. El compilador asigna **secciones**; el **linker**, guiado por el **linker script**, asigna **direcciones**.
6. Antes de `main`, el startup debe poner `.bss` en 0 y copiar `.data` de Flash a RAM.

🛠️ Flujo típico

reset → vector table (MSP y PC) → startup → inicialización de `.bss` y `.data` → `main()` → bucle infinito